

LAWGORITHM: AN AI-POWERED CASE–LAWYER MATCHING PLATFORM

Abdul Moeez

Muhammad Anas Amjad



DEPARTMENT OF COMPUTER SCIENCES
COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS
WAH CANTT – PAKISTAN
SESSION 2022-2026

LAWGORITHM: AN AI-POWERED CASE–LAWYER MATCHING PLATFORM

Undertaken By:

Abdul Moeez

Reg. No. CIIT/FA22-BCS-104/Wah

Muhammad Anas Amjad

Reg. No. CIIT/FA22-BCS-162/Wah

Supervised By:

Dr. Sheraz Anjum



A DISSERTATION SUBMITTED AS A PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF BACHELORS IN
COMPUTER SCIENCE

DEPARTMENT OF COMPUTER SCIENCES
COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS
WAH CANTT – PAKISTAN

FINAL APPROVAL

DEDICATION

We humbly dedicate this Final Year Project, *Lawgorithm*, to our esteemed supervisor, Dr. Sheraz Anjum, whose exceptional guidance, scholarly insight, and continuous encouragement have been instrumental in shaping this work. His profound knowledge, constructive feedback, and unwavering support throughout the development of this project not only enhanced the quality of our research but also inspired us to strive for excellence. His mentorship has played a significant role in strengthening our analytical thinking, problem-solving abilities, and professional growth.

We also dedicate this project to our collaborative efforts as a team. As two group members, we have worked with mutual respect, dedication, and a shared vision to transform our ideas into a meaningful and practical solution. This journey has been a valuable learning experience, teaching us the importance of teamwork, effective communication, time management, and perseverance in overcoming challenges.

Furthermore, we extend this dedication to our academic journey, which has equipped us with the knowledge, skills, and confidence required to undertake such a project. *Lawgorithm* stands as a reflection of our commitment, hard work, and passion for innovation, and it marks an important milestone in our academic and professional careers.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our respected supervisor, Dr. Sheraz Anjum, for his invaluable guidance, continuous support, and insightful feedback throughout the development of our Final Year Project, *Lawgorithm*. His expertise, patience, and encouragement greatly contributed to the successful completion of this project. We are truly thankful for the time and effort he invested in mentoring us and helping us overcome various challenges along the way.

We are also grateful to our respected faculty members of the Computer Science department for providing us with the knowledge and academic foundation necessary to undertake this project. Their teachings and support have played a significant role in shaping our skills and understanding.

Furthermore, we would like to acknowledge the support and cooperation between us as team members. Our collaboration, shared responsibilities, and dedication helped us stay focused and motivated throughout the project journey.

Lastly, we extend our heartfelt thanks to our families and friends for their constant encouragement, moral support, and understanding during the completion of this project. Their belief in us has been a source of strength and motivation.

This project would not have been possible without the support and contributions of all those who have directly or indirectly assisted us.

PROJECT BRIEF

PROJECT NAME:	Lawgorithm – AI-Based Lawyer Recommendation System
ORGANIZATION NAME:	COMSATS University Islamabad, Wah Campus
OBJECTIVE:	To develop an AI-powered system that matches clients with suitable lawyers based on case details using semantic analysis and rule-based scoring,
UNDERTAKEN BY:	Muhammad Anas Amjad & Abdul Moez
SUPERVISED BY:	Dr. Sheraz Anjum Professor Department of Computer Science COMSATS University Islamabad, Wah Campus
START DATE:	September 2025
Completed On:	April 2026
COMPUTER USED:	Intel Core i5 System, 8GB RAM, 256GB
SOURCE LANGUAGE:	Python 3.11, JavaScript (ES6), HTML5, CSS3
OPERATING SYSTEM:	Windows 11 (64-bit)
TOOLS USED:	Django 6.0.1, Django Channels, Daphne, MySQL (local), PostgreSQL/pgvector (Docker), Sentence-Transformers, scikit-learn, Google Gemini API (optional), Visual Studio Code, GitHub

ABSTRACT

Lawgorithm is an intelligent web-based platform designed to enhance the process of connecting clients with suitable legal professionals through the integration of artificial intelligence and structured decision-making techniques. Traditional lawyer directories in Pakistan primarily rely on manual search and basic filtering, which often leads to inefficient and inaccurate lawyer selection. This project addresses these limitations by introducing an automated and explainable lawyer recommendation system.

Lawgorithm uses a multi-stage hybrid matching pipeline. Client case details are first processed through structured filtering (practice area, city, and court), followed by semantic similarity computation using Sentence-Transformers (all-MiniLM-L6-v2, 384-dimensional embeddings) stored in PostgreSQL with pgvector.

A weighted hybrid scoring model combines semantic similarity with structured and quality signals such as experience, success rate, and review ratings. Additionally, an optional Large Language Model (Google Gemini 2.5 Flash) is used for candidate ranking and explanation generation. If unavailable, the system automatically falls back to embedding-based ranking to ensure reliability and consistency. The system also incorporates features such as secure user authentication, case management, lawyer profile verification, document handling, and an administrative dashboard for monitoring and control. Furthermore, the recommendation results are accompanied by explainable reasoning to ensure transparency and user trust.

Table of Contents

1. Introduction	12
1.1 System Introduction	12
1.2 Background of the System	12
1.3 Objectives of the System.....	13
1.4 Significance of the System.....	13
2. Overall Description	15
2.1 Product Perspective	15
2.2 Product Scope.....	16
2.3 Product Functionality	16
2.4 Users and Characteristics	16
2.5 Operating Environment.....	17
3. Specific Requirements	19
3.1 Functional Requirements	19
3.1.1 Authentication and User Account Management.....	19
3.1.2 Client Case Management	19
3.1.3 Lawyer Profile and Career History Management	20
3.1.4 AI-Based Case–Lawyer Matching System.....	20
3.1.5 Search, Filtering, and Browsing.....	22
3.1.6 Shortlisting, Invitations, and Interaction Handling	22
3.1.7 Administrator Management Functions	22
3.2 Behaviour Requirements	23
3.2.1 Overview	23
3.2.2 Textual Use-Case Descriptions	24
3.3 External Interface Requirements.....	28
3.3.1 User Interfaces	28
3.3.2 Hardware Interfaces	30
3.3.3 Software Interfaces.....	30
3.3.4 Communications Interfaces.....	30
4. Other Non-functional Requirements	31
4.1 Performance Requirements	31
4.2 Safety and Security Requirements	32

4.3 Software Quality Attributes.....	32
4.3.1 Reliability.....	32
4.3.2 Usability	33
4.3.3 Maintainability	33
4.3.4 Portability	33
4.3.5 Scalability.....	33
4.3.6 Interoperability.....	33
4.3.7 Robustness.....	33
5. Design Description.....	34
5.1 Composite Viewpoint.....	34
5.1.1 AI Matching Engine Architecture:	34
5.2 Logical Viewpoint.....	36
5.3 Information Viewpoint	37
5.4 Interaction Viewpoint.....	38
5.5 State Dynamics Viewpoint.....	42
5.6 Algorithm Viewpoint.....	46
5.7 Explainable Recommendation Component.....	48
6. Development and Tools.....	50
6.1 Introduction	50
6.2 Development Plan	50
6.3 Development Tools	53
6.4 Conclusion and Future Work / Extensions	53
6.5 Limitations and Risk Analysis	54
6.5.1 Limitations	54
6.5.2 Risk Analysis.....	55
7. Quality Assurance	56
7.1 Introduction	56
7.2 Traceability Matrix.....	56
7.3 Detailed Test Cases	57
7.4 Functional Requirements of Lawgorithm System.....	66
7.5 Requirement Traceability Matrix	68
7.6 Performance Evaluation	69

8. User Manual	72
8.1 Introduction	72
8.2 Hardware/Software Requirements for the System	72
8.3 Installation Guide for Application	73
8.4 Operating Manual	74

List of Figures

Figure 2.1: System Context Diagram	15
Figure 3.1: Use case diagram	24
Figure 5.1: Package Diagram	36
Figure 5.2: Class Diagram	37
Figure 5.3: ERD	38
Figure 5.4: Sequence Diagram for Case Intake and Preprocessing.	39
Figure 5.5: Sequence Diagram for Lawyer Profile Structuring and Verification.....	40
Figure 5.6: Sequence Diagram for the Core Matching Engine Flow.	40
Figure 5.7: Sequence Diagram for Fit Scoring and Explainability Generation	41
Figure 5.8: Sequence Diagram for Client Search and Filtering.	41
Figure 5.9: Sequence Diagram for Admin Oversight and Anomaly Detection.....	42
Figure 5.10: State Transition Diagram for Case Intake and Preprocessing	43
Figure 5.11: State Transition Diagram for Lawyer Profile Structuring	43
Figure 5.12: State Transition Diagram for the Match Request Process	44
Figure 5.13: State Transition Diagram for Fit Scoring and Explainability	44
Figure 5.14: State Transition Diagram for Case-Lawyer Interaction.....	45
Figure 5.15: State Transition Diagram for Admin Oversight and Analytics.....	45
Figure 5.16: Explainability Workflow in Lawyer Recommendation System.....	49
Figure 7.1: User registration	58
Figure 7.2: User login	59
Figure 7.3: Case submission	60
Figure 7.4: Case status tracking	61
Figure 7.5: Lawyers profile creation	62
Figure 7.6: Recommended lawyers	63
Figure 7.7: Lawyer verification.....	65
Figure 7.8: Recommendation display	66
Figure 8.1: User Registration and login	75
Figure 8.2: Client Operations	76
Figure 8.3: Lawyer Operations	77
Figure 8.4: Admin Dashboard	78
Figure 8.5: Search and filtering.....	79

List of Tables

Table 4.1: Performance Requirement Table	31
Table 5.1: State Dynamics Viewpoint	42
Table 6.1: Workload Distribution Table	51
Table 7.1: Requirement to Test Case Mapping	68
Table 7.2: Performance Results	71

1. Introduction

1.1 System Introduction

Lawgorithm is an AI-powered case–lawyer matching platform designed to assist clients in finding the most suitable legal professionals based on their case details, jurisdiction and budget. The system enables clients to submit case descriptions in English, which are then processed using text analysis models to extract relevant features. These features are matched against verified lawyer profiles, past matter summaries, and professional experience to generate a ranked list of recommended lawyers.

The platform provides a transparent “**Fit Score**” along with reasoning tags that explain why each lawyer is recommended—for example, similar past cases, relevant court expertise, or alignment with the client’s budget. Alongside recommendations, Lawgorithm includes functionalities such as profile verification, lawyer directory search, shortlisting, and consultation invitations, all delivered through an accessible Django-based web interface.

1.2 Background of the System

Existing online legal platforms in Pakistan function primarily as simple directories, offering only basic filtering options such as city and practice area. These platforms lack **intelligent recommendation features**, do not provide explainability for suggested lawyers, despite its necessity in the local legal context. Most systems also lack **verification mechanisms**, leading to user distrust and dependence on informal referrals.

Lawgorithm differs from these existing solutions by introducing **AI-driven matching**, multilingual processing, and explainable recommendations based on objective data such as lawyer career history, court experience, and practice areas. Unlike traditional platforms, Lawgorithm integrates region-specific metadata, lawyer verification workflows, and past matter embeddings, making it a **trustworthy solution** tailored for the Pakistani legal ecosystem.

1.3 Objectives of the System

The major objectives of the Lawgorithm system are:

- To collect case details from clients in English.
- To maintain structured lawyer profiles including **bar enrolment**, experience, and past matters.
- To authenticate and verify lawyer identities through an **admin-supervised workflow**.
- To generate **AI-based ranked lawyer recommendations** through similarity scoring and domain-specific rules.
- To provide clear reasoning tags explaining the rationale behind each recommendation.
- To enable clients to search, filter, shortlist, and invite lawyers for consultations.
- To provide an **admin dashboard** for monitoring, verification, and management of system lookups.
- To enhance transparency, trust, and efficiency in client–lawyer matching within Pakistan.

1.4 Significance of the System

Lawgorithm introduces transparency and structure into the largely unorganized legal-services discovery process in Pakistan. By combining verified lawyer data with explainable AI techniques, the system not only improves the accuracy of lawyer recommendations but also increases public trust in digital legal services.

The system supports various important application areas, including:

- Assisting individuals unfamiliar with legal procedures in quickly finding qualified lawyers.
- Providing **region-aware** and court-specific lawyer matching across Pakistan.

- Supporting law firms and independent lawyers in reaching relevant clients more efficiently.
- Allowing admins and regulatory bodies to monitor verification, compliance, and system activity.

2. Overall Description

2.1 Product Perspective

Lawgorithm is a new, self-contained software product designed to streamline the process of matching clients with suitable lawyers using **intelligent, explainable AI models**. It does not extend an existing product family nor replace a previous system; instead, it introduces an **innovative approach** in the Pakistani legal domain, where current platforms are limited to **basic lawyer directories** without intelligent recommendation, verification, or transparency. Lawgorithm integrates **data intake**, **AI-driven analysis**, and a **role-based user interface** into a single cohesive platform.

The software interacts with external components such as the underlying **database system**, **AI libraries** (transformers, FAISS), and Django's web framework, while presenting all functionalities to end-users through a **web-based interface**.

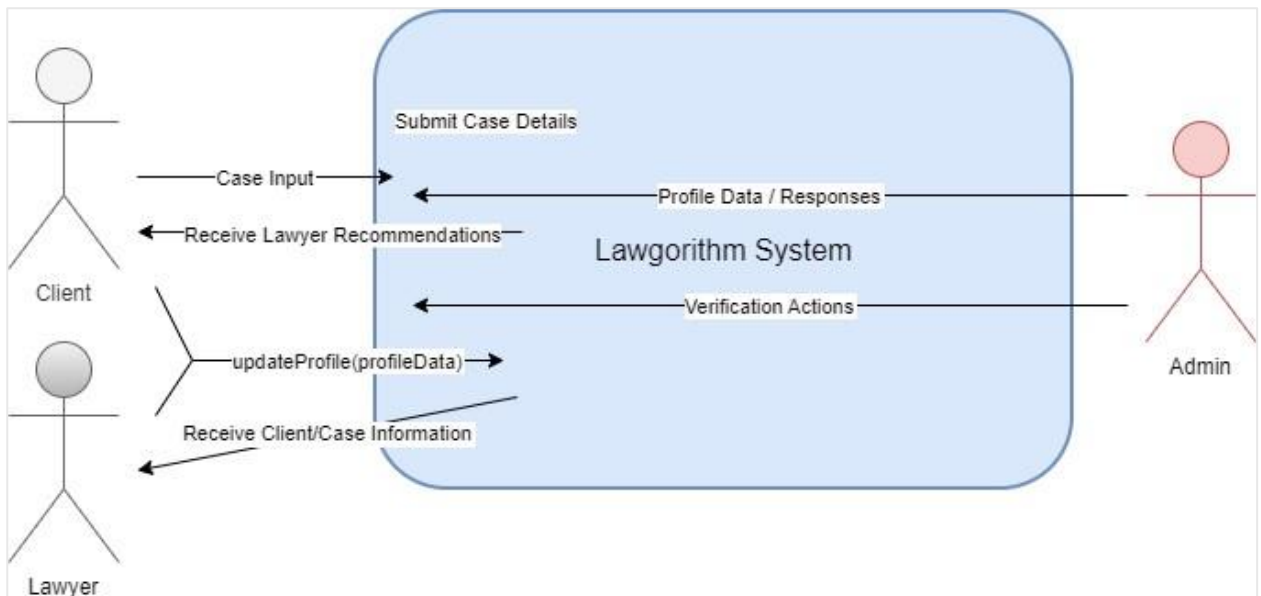


Figure 2.1: System Context Diagram

2.2 Product Scope

The current scope includes role-based portals (Client, Lawyer, Admin), case posting and tracking, AI-assisted lawyer recommendation, interaction workflow (invite/accept/reject/hire), document exchange, appointment scheduling, and post-case reviews. The platform also supports real-time communication and updates through WebSockets, including chat messages, interaction status updates, notifications, and online/offline presence. Admin scope includes lawyer verification and management of lookups (cities, courts, practice areas, fee bands).

2.3 Product Functionality

The major functionalities of Lawgorithm include:

- User registration, login, and role-based access control (Client/Lawyer/Admin)
- Email verification flow for account activation
- Client case posting, case tracking, and case detail management
- Hybrid lawyer recommendation with score and explainable reasons
- Client–lawyer interaction lifecycle: invited, accepted, rejected, hired
- Lawyer verification request and admin approval/rejection workflow
- Case-based chat and document sharing between client and lawyer
- Appointment scheduling for active cases
- Lawyer-driven case progress updates (status text, percentage, next hearing date)
- Ratings and reviews after case completion
- Admin management for cities, courts, practice areas, and fee bands

2.4 Users and Characteristics

Client User

- **Purpose:** To find a suitable lawyer for their case

- **Characteristics:** May have little or no technical background, uses the system occasionally, interacts with case forms, matched results, and lawyer details, requires highly understandable UI and bilingual support
- **Importance:** High — primary beneficiary of the recommendation engine

Lawyer User

- **Purpose:** To create and maintain their professional profile and receive client invitations
- **Characteristics:** Moderate computer literacy, uses system occasionally, must upload verification documents and complete profile details
- **Importance:** High — essential for generating accurate recommendations

Admin User

- **Purpose:** To verify lawyers, manage lookup tables, and oversee system integrity
- **Characteristics:** Technically proficient, uses system frequently, has highest privilege level
- **Importance:** Critical — ensures platform trust, compliance, and data correctness

System-Level Distinction

- Most important users: **Clients & Admin**
- Secondary users: **Lawyers**

2.5 Operating Environment

Lawgorithm will operate in a **web-based environment** using the Django framework, supporting modern browsers such as Chrome, Edge, and Firefox. The application is designed to be deployed on a local machine, following restrictions for the Final Year Project. The backend requires **Python 3.11+**, Django 5, and a **relational database** such as

MySQL or PostgreSQL. The AI components depend on libraries like sentence-transformers, FAISS, scikit-learn, and numpy.

Minimum Platform Requirements:

- **Hardware:** Dual-core processor, minimum 8 GB RAM, 5 GB free disk space
- **Software:** Windows 10/11 or Linux (Ubuntu recommended), Python 3.11+, Django 5, MySQL/PostgreSQL, Chrome/Edge/Firefox

The system must coexist with background processes such as **web servers, local databases,** and Python runtime environments. No external hardware or SMS gateway is required.

3. Specific Requirements

3.1 Functional Requirements

The functional requirements of the system are organized into major functional areas. Each area contains detailed requirements that describe the core operations the system must perform.

3.1.1 Authentication and User Account Management

1. User Registration Functionality

The system shall allow clients, lawyers, and administrators to create accounts by providing essential information such as name, email, password, and role type.

2. Secure Login and Session Management

The system shall authenticate users using email and password and maintain secure sessions throughout user interaction.

3. Role-Based Access Control

The system shall restrict access to features based on user roles (Client, Lawyer, Admin), ensuring that each role can only access its designated functionalities.

4. Profile Completion Monitoring

The system shall notify users to complete their required profile sections before accessing certain features (e.g., lawyers must add their practice details).

3.1.2 Client Case Management

1. Case Submission Functionality

The system shall allow clients to submit case details including category, subcategory, court level, location, budget band, urgency, language preference, and description.

2. Case Document Upload

The system shall allow clients to upload optional supporting documents with their case.

3. Case Status Tracking

The system shall maintain predefined case statuses such as Draft, Submitted, Matched, Shortlisted, Hired, and Closed, and update the status as actions progress.

4. Case Editing and Review

The system shall allow clients to review and edit draft cases before submission.

3.1.3 Lawyer Profile and Career History Management

1. Profile Information Management

The system shall allow lawyers to add and update their credentials including bar enrolment number, courts, languages, city, province, years of experience, and fee style.

2. Career History Recording

The system shall allow lawyers to record past matters including practice area, subarea, court details, year, duration, fee band, and matter summary.

3. Verification Document Submission

The system shall allow lawyers to upload verification documents that require administrator approval.

3.1.4 AI-Based Case–Lawyer Matching System

1. Candidate Retrieval:

The system filters only approved lawyers and prioritizes candidates based on structured attributes in the following order:

- Subcategory + City + Court
- Subcategory + City
- Subcategory + Court
- Subcategory only

- City or Court only

A configurable limit is applied to restrict the candidate pool.

2. Semantic Embedding Matching:

Case descriptions and lawyer profiles are converted into 384-dimensional embeddings using Sentence-Transformers (all-MiniLM-L6-v2). Similarity is computed using cosine similarity via PostgreSQL pgvector.

3. Hybrid Scoring:

A weighted hybrid scoring model combines semantic similarity with structured and quality-based signals including:

- Practice area match
- Court match
- City proximity
- Experience years
- Success rate
- Review ratings

4. Optional LLM-Based Ranking:

If configured, Google Gemini (2.5 Flash) is used to rank candidates and generate structured outputs (lawyer_id, score, reasons). The system uses low temperature settings for deterministic results.

5. Fallback Strategy:

If the LLM service is unavailable or fails, the system automatically falls back to embedding-based hybrid scoring.

6. Explainability:

Each recommendation includes 2–3 human-readable reasons, optionally enhanced using Gemini for clarity and professionalism.

3.1.5 Search, Filtering, and Browsing

1. Lawyer Directory Search

The system shall allow users to search lawyers by keywords, names, or practice details.

2. Filtering Functionality

The system shall allow filtering by city, court, practice area, language, and fee band.

3. Lawyer Detail View

The system shall allow users to view complete lawyer profiles including verification status and career history.

3.1.6 Shortlisting, Invitations, and Interaction Handling

The system maintains interaction records between a case and lawyer using the following statuses: **invited**, **accepted**, **rejected**, and **hired**. Clients can send invitations from recommendation results, lawyers can accept/reject and optionally propose fees, and clients can finalize hiring from accepted interactions. These interaction transitions are reflected in real-time through WebSocket status events.

3.1.7 Administrator Management Functions

- 1. Verification Management:** Admin reviews pending lawyer profiles/documents and approves or rejects requests.
- 2. Lookup Management:** Admin manages cities, courts, practice areas/categories, and fee bands.
- 3. Dashboard Oversight:** Admin dashboard provides counts for total/pending/approved lawyer profiles and total cases.
- 4. Notification Handling:** Admin can open and mark relevant notification flows linked to pending verification actions.

3.2 Behaviour Requirements

3.2.1 Overview

The behavioural requirements describe how the system behaves from a user-interaction perspective. These requirements define all services, tasks, and functions performed by the system as seen by external actors. The behaviour is represented through a textual use-case description, covering all primary actors and their interactions with the Lawgorithm platform.

The system includes three main actors:

1. Client – Submits cases, views recommendations, shortlists, and invites lawyers.
2. Lawyer – Manages profile, adds career history, and responds to invitations.
3. Admin – Verifies lawyers, manages system lookups, and performs oversight.
4. AI Matching Engine (System Actor) – Internal automated actor that processes case data and generates lawyer recommendations.

The following subsections provide a textual representation of all use cases, their purpose, and the flow involved.

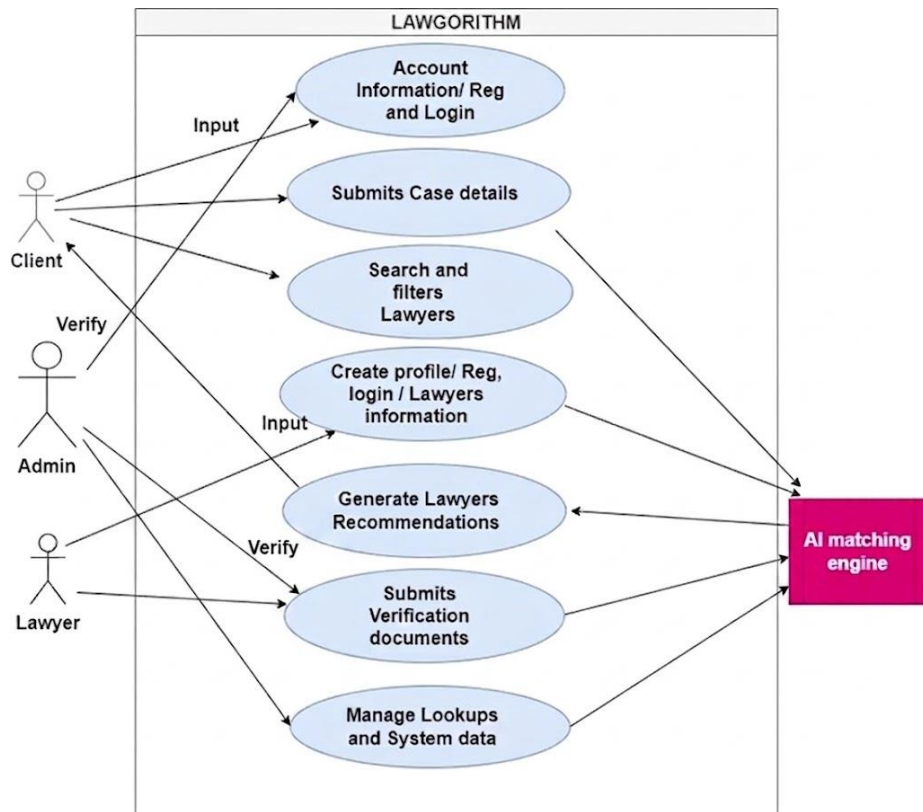


Figure 3.1: Use case diagram

3.2.2 Textual Use-Case Descriptions

Use Case 1: User Registration and Login

Actors: Client, Lawyer

Purpose: Allow users to create accounts and securely authenticate into the system.

Description:

- The user provides email, name, and password.
- The system validates information and creates a new account.
- During login, credentials are verified, and role-based dashboard is loaded.
- Outcome: User successfully accesses the platform according to their assigned role.

Use Case 2: Submit Case Details

Actor: Client

Purpose: Allow clients to describe their legal matter for processing.

Description:

- The client fills in required case fields (category, court level, city, fee band, description, urgency, language preference).
- Optional documents may be uploaded.
- The system stores the case and sets its initial status to “Submitted.”
- Outcome: Case is saved and sent for matching.

Use Case 3: Generate Lawyer Recommendations

Actors: Client, AI Matching Engine

Purpose: Provide ranked lawyer recommendations for a specific case.

Description:

- The AI engine preprocesses the case (cleaning, translation, embedding).
- Lawyer embeddings are retrieved from PostgreSQL using the pgvector extension with cosine similarity queries. Similarity scores are computed and combined with rule-based features.
- Fit Score (0–100) and explanation reasons are generated.
- Ranked results are displayed to the client.
- Outcome: Client receives a structured list of recommended lawyers with reasoning.

Use Case 4: Search and Filter Lawyers

Actor: Client

Purpose: Allow users to manually search and browse lawyers.

Description:

- The client applies filters such as city, practice area, court, or language.
- The directory updates results in real time.
- Keyword search retrieves matching profiles using TF-IDF/ElasticSearch.
- Outcome: Client finds relevant lawyers even without submitting a case.

Use Case 5: Shortlist and Invite Lawyers

Actor: Client

Purpose: Enable clients to directly interact with potential lawyers.

Description:

- The client adds lawyers to a shortlist.
- An invitation to consult can be sent.
- Interaction events are recorded (invited, accepted, hired)
- Outcome: Lawyer-client engagement is tracked for future reference.

Use Case 6: Manage Lawyer Profile

Actor: Lawyer

Purpose: Allow lawyers to maintain and update their professional details.

Description:

- Lawyer updates basic information: council, experience, language, fee style.
- Lawyer adds practice areas and previous matters with summaries.
- Profile completeness meter updates automatically.
- Outcome: A structured lawyer profile becomes available for matching and search.

Use Case 7: Submit Verification Documents

Actor: Lawyer

Purpose: Support secure validation of professional credentials.

Description:

- Lawyer uploads bar enrolment certificate or other verification documents.
- System stores and forwards documents to Admin for review.
- Outcome: Lawyer enters “Pending Verification” state.

Use Case 8: Verify Lawyer Accounts

Actor: Admin

Purpose: Ensure authenticity of lawyer profiles.

Description:

- Admin reviews uploaded documents.
- Admin accepts or rejects verification with comments.
- Upon approval, a “Verified” badge appears on the lawyer’s profile.
- Outcome: Verified lawyers receive priority in matching results.

Use Case 9: Manage Lookups and System Data

Actor: Admin

Purpose: Maintain full platform control.

Description:

- Admin manages courts, cities, case categories, fee bands, languages.
- Admin imports or exports CSV data.
- Admin views analytics such as number of cases, verifications, or matches.

- Outcome: The system remains accurate and up to date.

Use Case 10: Rate Lawyer Interactions (Optional)

Actor: Client

Purpose: Allow feedback and quality assurance.

Description:

- After hiring a lawyer, the client leaves a star rating and comments.
- System updates the lawyer's average rating.
- Outcome: Lawyer performance influences future recommendations.

Use Case 11: System-Level Matching and Preprocessing

Actor: AI Matching Engine

Purpose: Internally process case data.

Description:

- AI cleans text, translates if needed, and generates embeddings.
- NER extracts legal entities from lawyer experience summaries.
- Matching scores and explainability tags are computed.
- Outcome: Automated analysis provides intelligent recommendations.

3.3 External Interface Requirements

3.3.1 User Interfaces

Lawgorithm will provide the following user interfaces:

Client UI

- Login/Registration Screen
- Dashboard (“My Cases”)
- Create Case Form
- AI Matching Results Page (ranked lawyers with Fit Score)
- Shortlist Page
- Invite & Message Page

Lawyer UI

- Profile Management Screen
- Add/Edit Practice Areas & Matters
- Verification Upload Screen
- Invitation Response Screen

Admin UI

- Verify Lawyer Panel
- Lookup Management Screen
- CSV Import/Export Screen
- Analytics Dashboard

Each UI includes:

- Standard buttons: Save, Cancel, Edit, Delete, Submit
- Validation messages
- Error banners for incorrect input

3.3.2 Hardware Interfaces

(This project uses only software—no hardware.)

- Runs on standard computing hardware (desktop/laptop).
- Requires no sensors, devices, or external hardware APIs.

3.3.3 Software Interfaces

The system interfaces with the following software components:

- Operating System: Windows or Linux
- Backend Framework: Django 5
- Database: MySQL or PostgreSQL
- ML Libraries: Sentence-Transformers, scikit-learn, PostgreSQL pgvector
- Frontend: Bootstrap (Django Templates)

APIs exchanged:

The platform primarily uses Django server-rendered templates and form submissions for web interactions. Persistent data operations are performed through Django ORM. Real-time features (chat, notifications, interaction status, and presence updates) are implemented through Django Channels WebSocket interfaces.

3.3.4 Communications Interfaces

- Web communication is handled through HTTP/HTTPS protocols.
- All data exchanged between client and server shall be encrypted using TLS.
- No external APIs or third-party integrations except optional email.

4. Other Non-functional Requirements

4.1 Performance Requirements

The performance requirements ensure that the system delivers timely responses, maintains acceptable throughput, and provides a smooth experience for clients, lawyers, and administrators. Considering the nature of the platform—text processing, AI-based matching, and data retrieval—the following performance constraints have been defined:

Performance Requirements Table:

Table 4.1: Performance Requirement Table

Requirement	Description
System Response Time	All standard UI page load operations (login, dashboard, listing pages) should respond within 2 seconds under normal operating conditions.
AI Matching Processing Time	Generating lawyer recommendations based on a case submission should take no more than 4–6 seconds , including text embedding, vector search, and fit-score computation.
Search Filter Response	Applying filters or keyword searches on lawyer directory should update results within 1 second .
Database Transaction Time	Critical transactions such as case creation, profile update, or verification approvals should not exceed 2 seconds .
Concurrent User Handling	The system should comfortably handle 50–100 simultaneous users on typical academic-deployment hardware.
Model Loading Time	AI model initialization (embedding model + FAISS index) should load within 8–10 seconds during server startup.

4.2 Safety and Security Requirements

The system deals with sensitive legal data submitted by clients and professional information uploaded by lawyers. To protect this data and prevent misuse, several safety and security mechanisms are required.

Safety Requirements

1. The system shall validate and sanitize all user inputs to avoid malicious data injection.
2. Uploaded documents (verification files, attachments) shall be scanned for harmful file types.
3. Access to AI modules must fail safely. If the AI model is unavailable, the system shall revert to a fallback rule-based matching method.

Security Requirements

- Passwords are securely stored using Django's built-in hashing mechanisms.
- Role-based access restrictions are enforced at view/route level.
- Authenticated sessions are required for protected user operations.
- Verification and case actions are restricted by ownership and role checks.
- Transport security (HTTPS/TLS) is deployment-dependent and must be enforced in production configuration.
- File upload validation and advanced malware scanning can be listed as future hardening work if not currently integrated.

These security measures ensure confidentiality, integrity, and availability of user data.

4.3 Software Quality Attributes

This section describes the qualitative characteristics that the Lawgorithm system must exhibit. Each attribute includes a description of how it will be ensured.

4.3.1 Reliability

- The system should remain functional for extended periods without crashes.
- Regular automatic backups of the database should be scheduled.

- Fault tolerance shall be applied in AI modules with fallback matching.

4.3.2 Usability

- User interfaces will follow simple and intuitive layouts using Bootstrap.
- Clear navigation flows for clients, lawyers, and admins.

4.3.3 Maintainability

- Modular development using Django apps ensures easy updates and modifications.
- All core components (intake, matching, profiles) are separated into independent modules.
- Code will follow consistent documentation and naming conventions.

4.3.4 Portability

- The system should run on any OS supporting Python (Windows, Linux, macOS).
- Database independence is maintained between MySQL and PostgreSQL.

4.3.5 Scalability

- FAISS indexing enables fast vector similarity searches, scalable up to thousands of profiles.
- Addition of new courts, cities, or case categories does not impact system performance.

4.3.6 Interoperability

- REST-based communication enables future integration with external systems.
- Data structures follow standard JSON formatting.

4.3.7 Robustness

- System should handle unexpected shutdowns without data corruption.
- Invalid operations must return meaningful error messages.

5. Design Description

5.1 Composite Viewpoint

This viewpoint outlines the major structural constituents of the system and illustrates how various subsystems integrate to form the complete software solution. Lawgorithm is composed of the following subsystems:

- **User Management Subsystem** (clients, lawyers, admins)
- **Case Management Subsystem**
- **Lawyer Profile & Verification Subsystem**
- **AI Matching Subsystem**
- **Search & Directory Subsystem**
- **Admin Panel & Lookups Subsystem**

These subsystems operate collaboratively to provide the end-to-end matchmaking flow. The system is deployed on a local machine running Django, where the backend, AI services, and database operate as a unified software stack.

5.1.1 AI Matching Engine Architecture:

The AI Matching Engine in Lawgorithm follows a modular and production-oriented architecture consisting of the following components:

1. Embedding Pipeline:

Responsible for generating vector representations of lawyer profiles and case descriptions using Sentence-Transformers. Embeddings are stored in PostgreSQL using pgvector.

2. Matching Service:

Handles candidate retrieval, ranking, and orchestration of the matching workflow.

3. Hybrid Scoring Module:

Combines semantic similarity with structured and quality signals to produce a final Fit Score.

4. LLM Ranking Module:

Uses Google Gemini (2.5 Flash) for advanced ranking and explanation generation when available.

5. Auto-Refresh Mechanism:

Automatically updates embeddings when lawyer profiles are modified or approved.

6. Caching Layer:

Stores recent match results to improve performance and reduce recomputation.

7. Evaluation Pipeline:

Supports model evaluation using metrics such as Recall@K and Mean Reciprocal Rank (MRR), enabling continuous improvement.

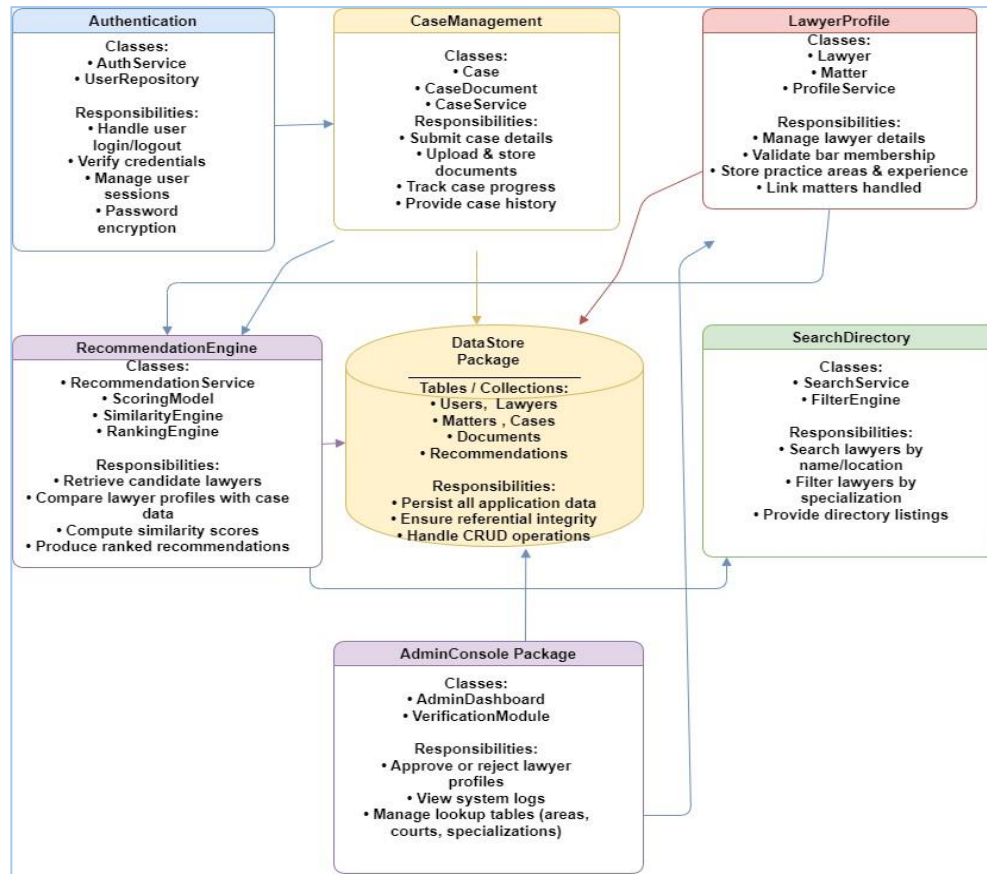


Figure 5.1: Package Diagram

5.2 Logical Viewpoint

This viewpoint focuses on the static structure of the system in terms of classes, interfaces, and their relationships. The core logical components include:

- **User Class** — Represents clients, lawyers, and administrators with shared attributes.
- **Lawyer Class** — Extends user information with professional details.
- **Case Class** — Stores client case submissions.
- **Matter Class** — Contains previous cases handled by lawyers.
- **Verification Class** — Manages lawyer verification workflow.
- **Interaction Class** — Manages shortlisting, invitations, and hiring events.

- **MatchingEngine Class** — Implements embedding generation and similarity calculations.

These classes form the backbone of the system architecture.

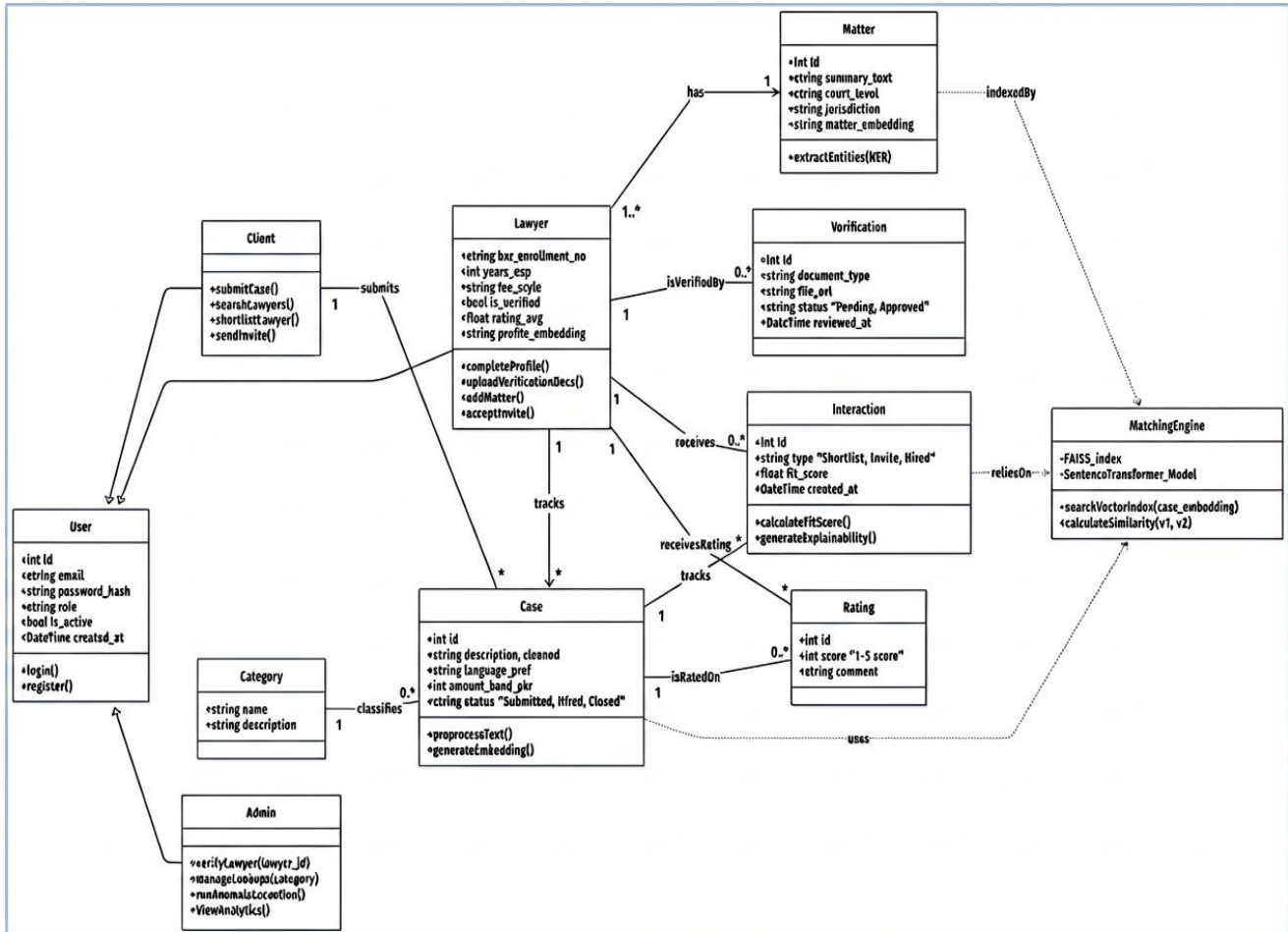


Figure 5.2: Class Diagram

5.3 Information Viewpoint

This viewpoint focuses on how persistent data is structured and stored. Lawgorithm uses a relational database to maintain:

- User information
- Lawyer credentials and history

- Cases and case statuses
- Lawyer–client interactions
- Verification data
- Rating records

The database schema is normalized and ensures referential integrity among all tables.

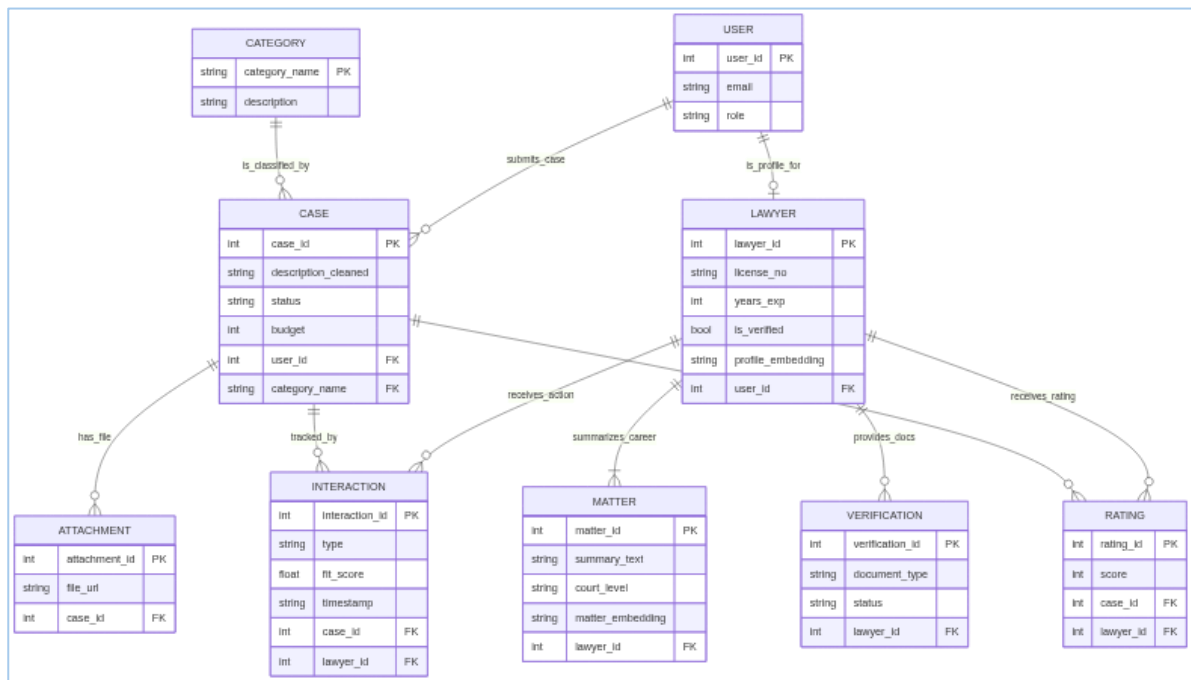


Figure 5.3: ERD

5.4 Interaction Viewpoint

This describes how objects interact to fulfill system requirements. Key interactions include:

- **Case Submission Interaction**

Client submits case → system validates → stores case → triggers matching engine.

- **AI Matching Interaction**

Case description → embedding model → vector search → fit-score computation → ranked output.

- **Verification Interaction**

Lawyer uploads documents → admin reviews → system updates verification status.

- **Shortlisting Interaction**

Client shortlists lawyer → interaction record created → lawyer notified.

These interactions collectively enable the end-to-end functionality.

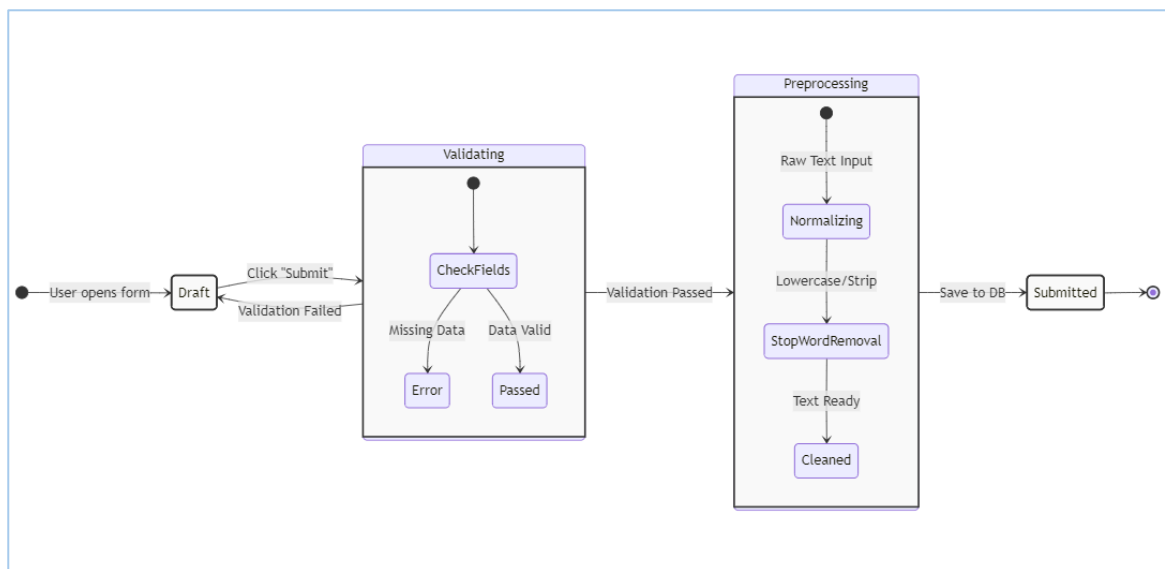


Figure 5.4: Sequence Diagram for Case Intake and Preprocessing.

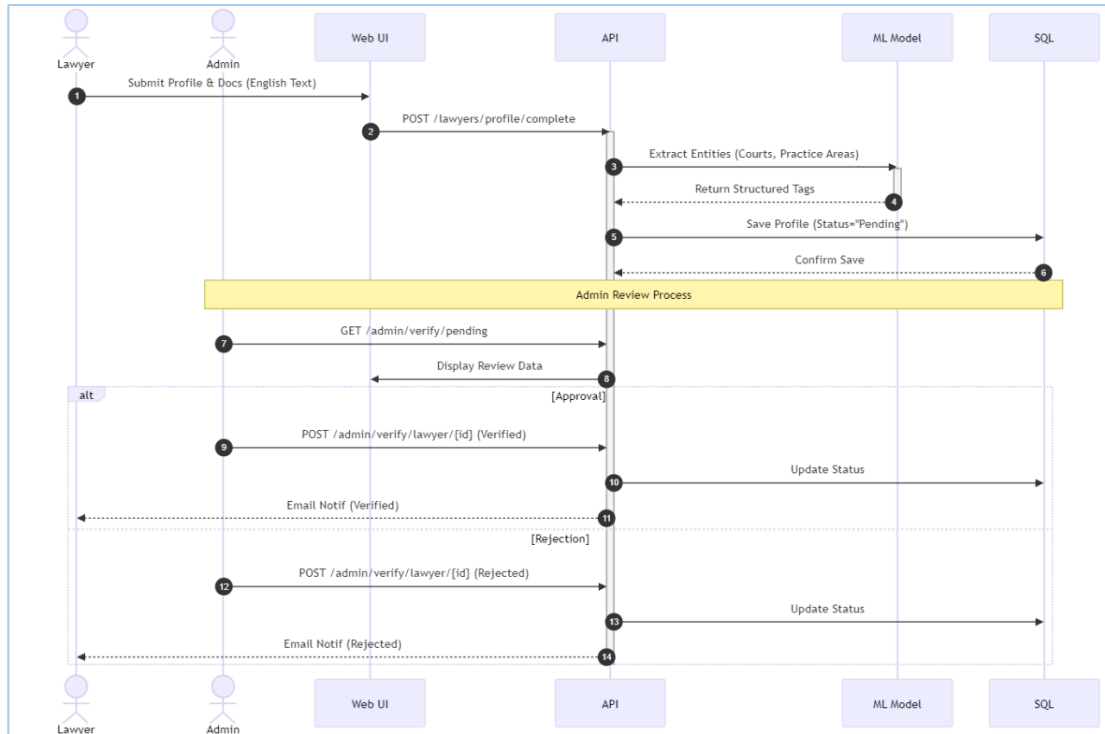


Figure 5.5: Sequence Diagram for Lawyer Profile Structuring and Verification

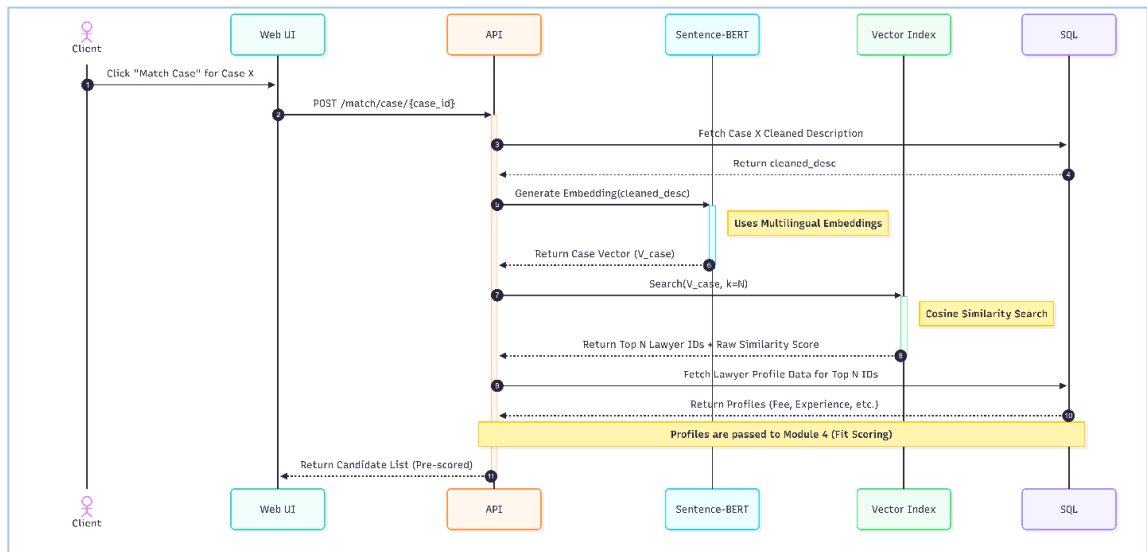


Figure 5.6: Sequence Diagram for the Core Matching Engine Flow.

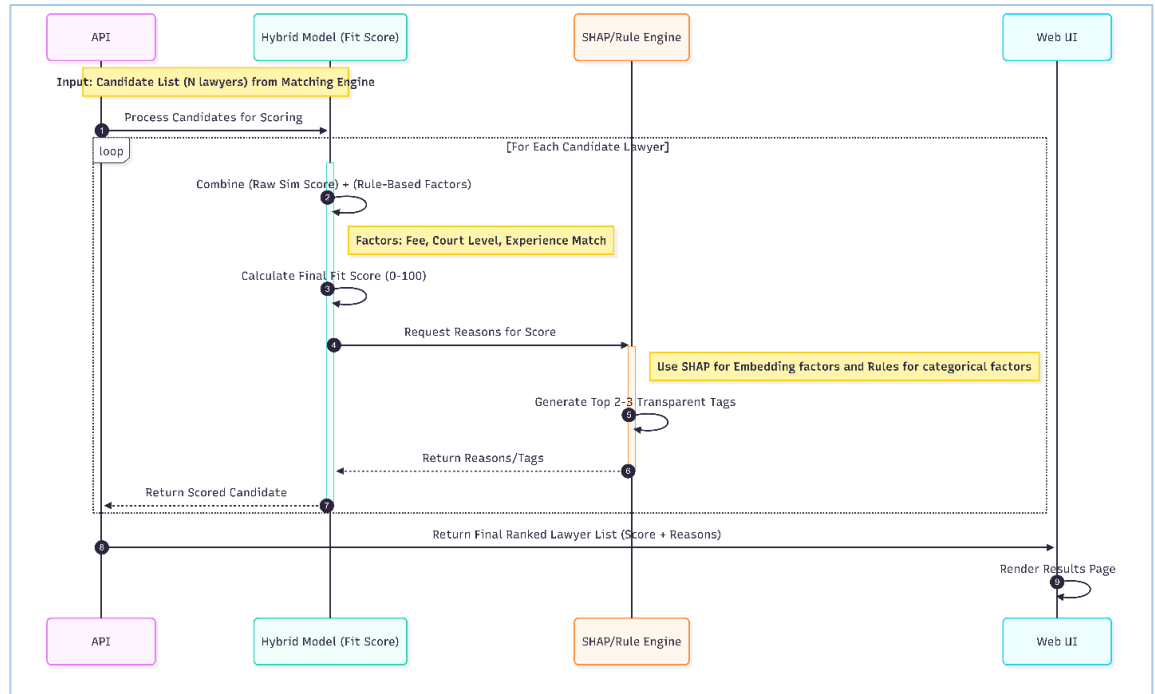


Figure 5.7: Sequence Diagram for Fit Scoring and Explainability Generation

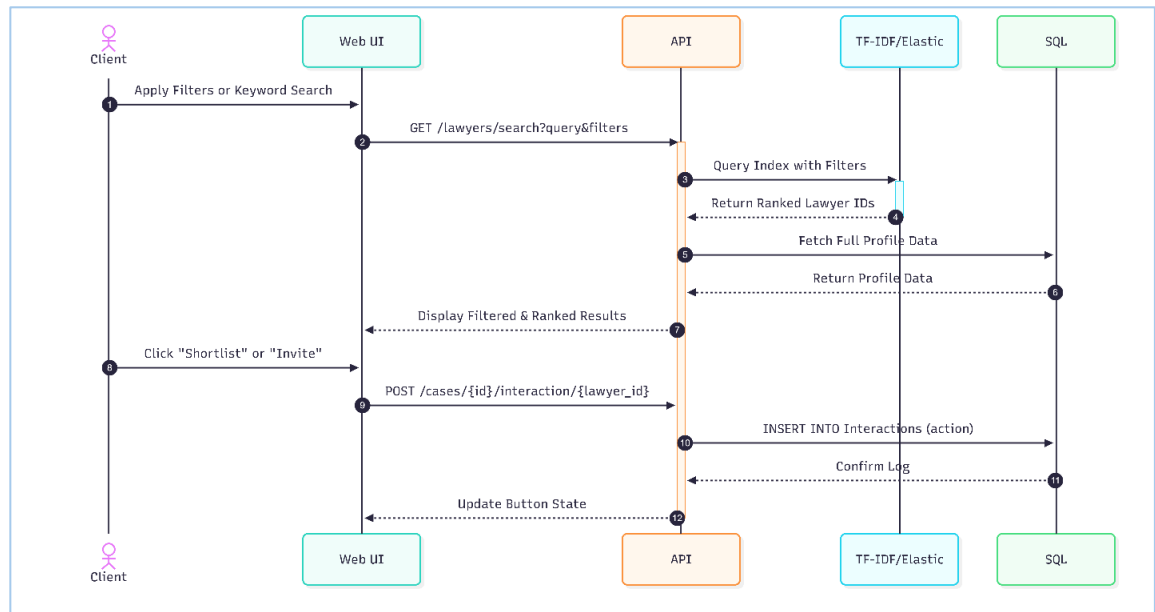


Figure 5.8: Sequence Diagram for Client Search and Filtering.

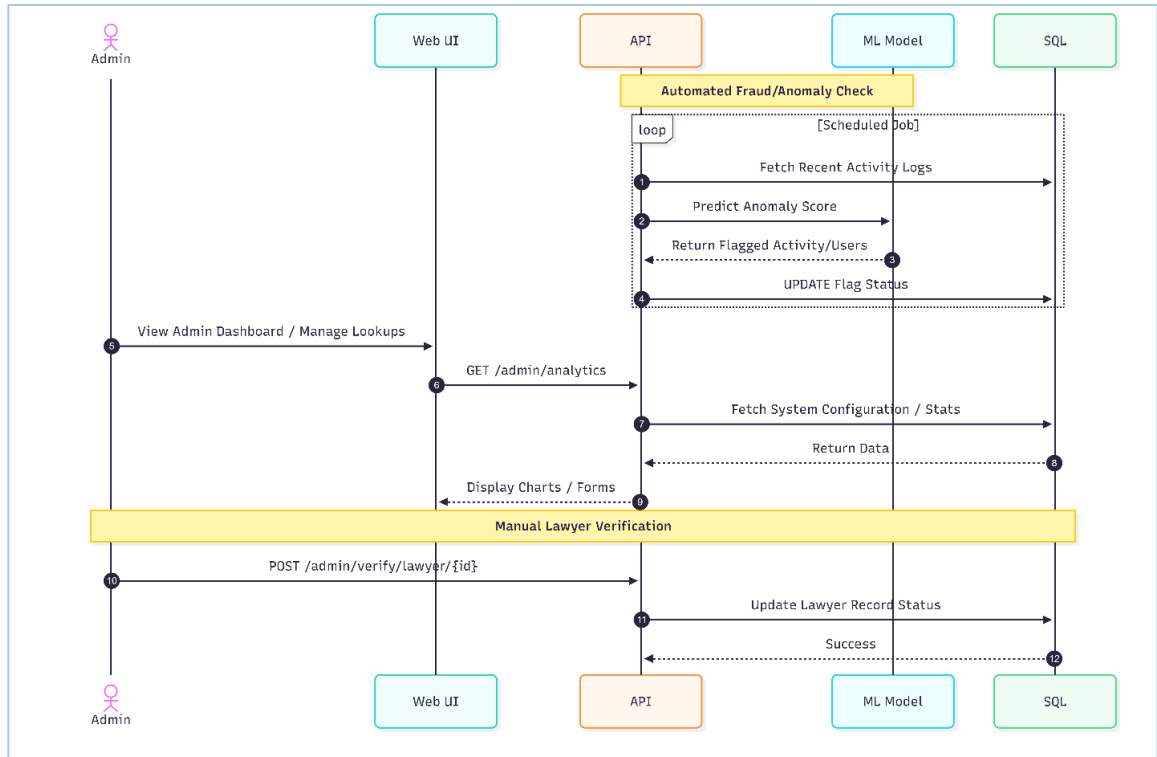


Figure 5.9: Sequence Diagram for Admin Oversight and Anomaly Detection.

5.5 State Dynamics Viewpoint

This viewpoint explains the dynamic state transitions of major system entities.

Table 5.1: State Dynamics Viewpoint

State	Trigger/Action	Next State
Draft	Client submits case	Submitted
Submitted	AI matching completes	Matched
Matched	Client shortlists lawyer	Shortlisted
Shortlisted	Lawyer accepts	Hired
Hired	Case is completed	Closed

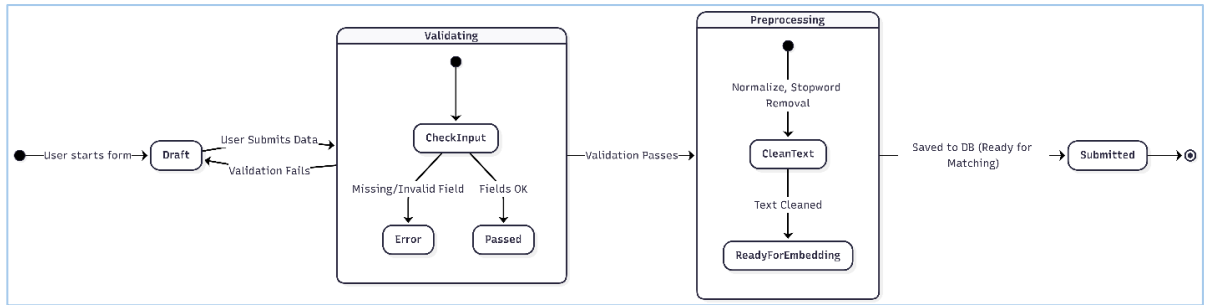


Figure 5.10: State Transition Diagram for Case Intake and Preprocessing

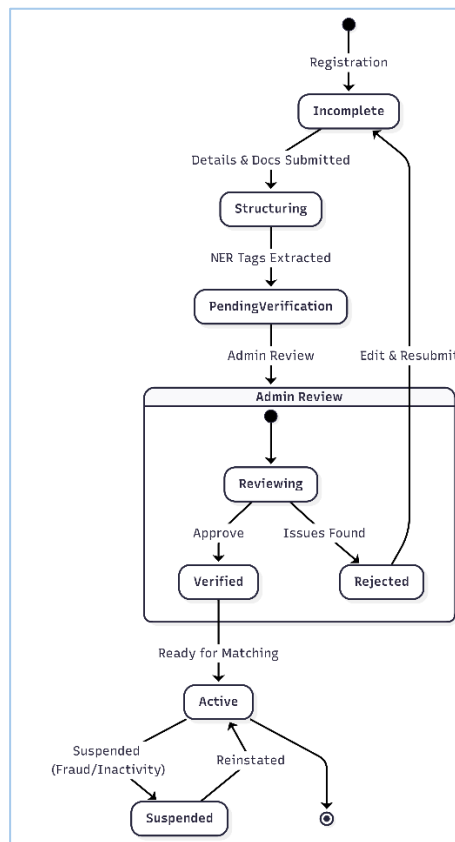


Figure 5.11: State Transition Diagram for Lawyer Profile Structuring



Figure 5.12: State Transition Diagram for the Match Request Process

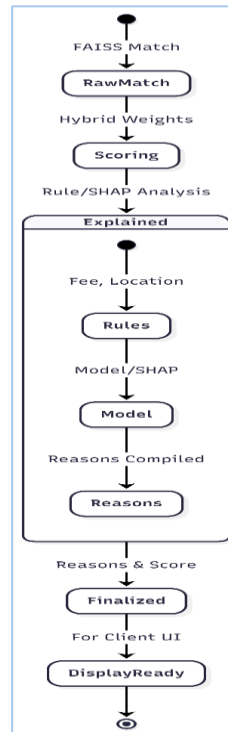


Figure 5.13: State Transition Diagram for Fit Scoring and Explainability

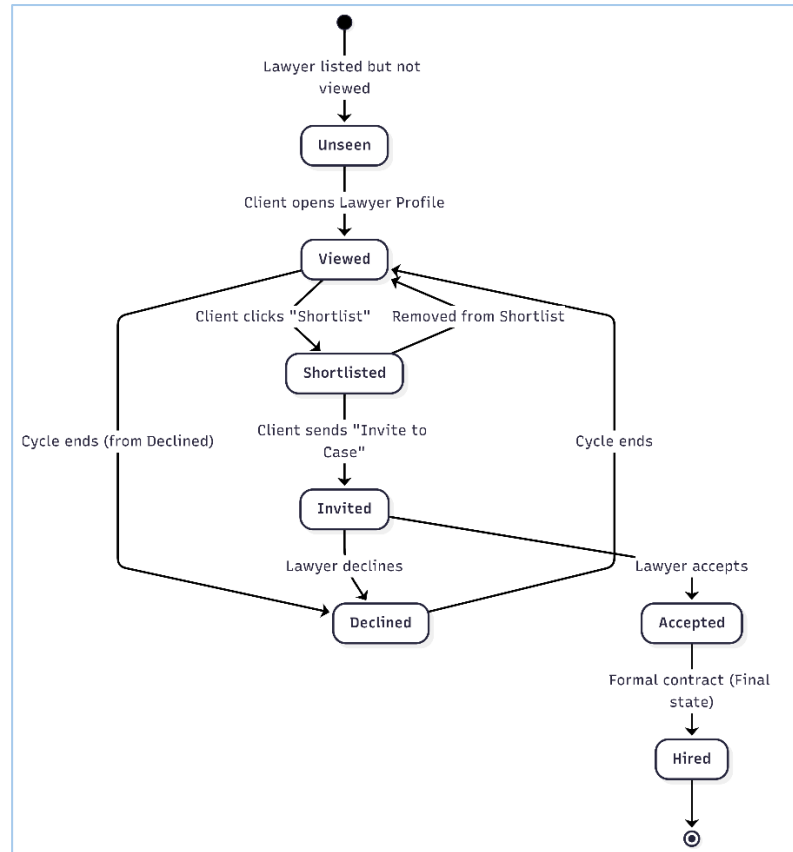


Figure 5.14: State Transition Diagram for Case-Lawyer Interaction

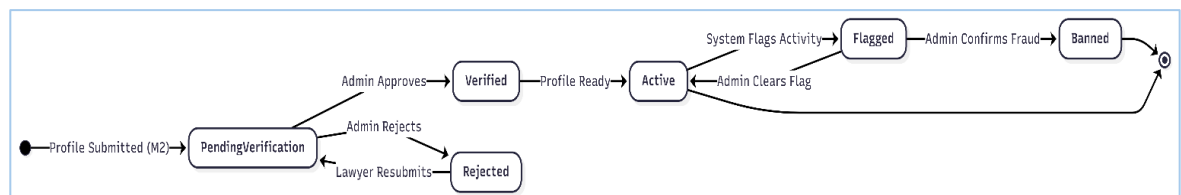


Figure 5.15: State Transition Diagram for Admin Oversight and Analytics

5.6 Algorithm Viewpoint

The Algorithm Viewpoint provides a detailed description of the internal logic, operations, and decision-making processes implemented within the system’s core components. It focuses on how individual functions, methods, and modules perform their tasks, how inputs are transformed into outputs, and how logical flows ensure consistent and reliable system behavior. This viewpoint acts as the bridge between the system’s functional requirements and its technical implementation, ensuring that each operation is fully understood before coding begins.

For Lawgorithm—an AI-powered case-lawyer matching platform—the algorithmic logic is central to the system’s objectives, particularly in areas such as case preprocessing, embedding generation, similarity matching, Fit Score calculation, explainability reasoning, and workflow state transitions. The algorithms define how client-submitted case details are interpreted, how lawyer expertise is evaluated, and how the system determines the most suitable matches.

Since the system does not rely on hardware logic or embedded systems and instead depends heavily on computation, text processing, and machine learning pipelines, **Pseudo Code** is the most appropriate method for representing the algorithmic viewpoint. Unlike decision tables (which are best suited for finite, rule-based scenarios), pseudo code allows for representing sequential processes, iterative operations, conditional logic, and AI model execution, all of which are required for modules such as natural language preprocessing, vector similarity computation, and ranking.

Explanation of Algorithm Components

1. Text Preprocessing

Cleans and normalizes English case text using multilingual tokenization, translation (if needed), and removal of noise.

2. Embedding Generation

Uses Sentence-Transformers (all-MiniLM-L6-v2) to convert text into 384-dimensional semantic vectors.

3. Similarity Computation

Using cosine similarity computed via PostgreSQL pgvector extension at database level.

4. Rule-Based Evaluation

Applies domain-specific rules such as:

- City/province
- Language
- Matching practice area
- Matching court type
- Relevance
- Compatibility
- Fee band alignment

5. Fit Score Aggregation

Combines AI similarity and rule-based values to produce a 0–100 Fit Score.

6. Explainability Generation

Produces 2–3 human-understandable reasons such as:

“Handled 12 similar cases”, “Practices in the same court”.

7. Sorting and Output

Generates the final ranked list of recommended lawyers.

8. Advanced Hybrid Scoring Mechanism:

The system applies a weighted scoring model combining multiple signals:

- Semantic similarity: 58%
- Practice area match: 10%
- Court match: 7%
- City match: 6%
- Experience: 3%
- Success rate: 9%
- Review quality: 5%
- Review confidence: 2%

9. Cold-Start Handling:

Bayesian smoothing is applied to success rates and review scores to prevent unfair penalization of new lawyers.

10. Fallback Mechanism:

If LLM-based ranking fails, the system falls back to embedding-based ranking to ensure deterministic and reliable results.

5.7 Explainable Recommendation Component

The Lawgorithm system includes an explainable AI component to enhance transparency and user trust. Instead of only presenting ranked lawyer results, the system provides clear, human-understandable reasons for each recommendation.

After computing hybrid scores using semantic similarity and structured features, the system generates explanation statements based on key factors such as practice area, court, location, and experience. Each recommended lawyer is accompanied by 2–3 concise justifications.

Example explanations include:

- “Specializes in the same legal category as the case”

- “Practices in the same court and city”
- “Has handled similar cases successfully”

If the AI-based module (Google Gemini) is unavailable, the system uses rule-based explanations to ensure consistent output.

This component improves interpretability and helps users make informed decisions.

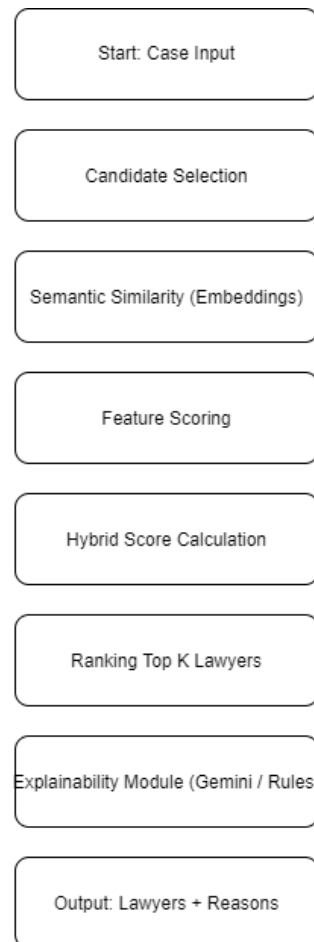


Figure 5.16: Explainability Workflow in Lawyer Recommendation System

6. Development and Tools

This chapter presents a comprehensive overview of the development process of the Lawgorithm system. It outlines the overall development strategy, planning, and task distribution among team members. Furthermore, it highlights the tools and technologies used during the implementation phase. The chapter also provides insights into the project's progress across two academic semesters and concludes with a discussion on system achievements and potential future enhancements.

6.1 Introduction

This chapter describes the development methodology, tools, and technologies used in building the Lawgorithm system. It explains how the project was planned and executed in phases across two semesters. Additionally, it highlights the distribution of tasks among team members and the tools utilized for development, testing, and deployment.

6.2 Development Plan

The Lawgorithm project was developed by a team of two members:

1. **Muhammad Anas Amjad**
2. **Abdul Moeez**

Development Phases

Phase 1 – 7th Semester (30% Completion)

The initial phase focused on building the foundational structure of the system. The following components were completed:

- Frontend UI design (Client, Lawyer, and Admin interfaces)
- User Authentication System (Login/Signup)
- Role-based access control (Client, Lawyer, Admin)

- User Profile Management (Client & Lawyer)
- Basic CRUD Operations (Cases, Profiles)
- Initial Database Schema Design
- Static Lawyer Directory (without AI)

Phase 2 – 8th Semester (70% Completion)

The second phase focused on integrating intelligent features and completing the system:

- AI-Based Recommendation Engine (core module)
- NLP-based case processing using Sentence Transformers
- Vector similarity search using FAISS
- Fit Score calculation (hybrid model)
- Explainable recommendations (reason tags)
- Case lifecycle management (state transitions)
- Search & filtering system (advanced directory)
- Lawyer verification system (Admin module)
- Document upload and handling
- System testing and optimization
- Final UI/UX improvements

Workload Distribution

Table 6.1: Workload Distribution Table

Task	Assigned To
Frontend Design	Both Members

Backend Development (Django)	Abdul Moeez
Database Design	Both Members
Authentication & Security	Muhammad Anas
AI Matching Engine	Abdul Moeez
Testing & Debugging	Abdul Moeez
Documentation	Muhammad Anas

Gantt Chart (Tabular Form)

Task	Duration	Semester
Requirement Analysis	2 Weeks	7th
UI/UX Design	3 Weeks	7th
Authentication System	2 Weeks	7th
CRUD Operations	3 Weeks	7th
Database Design	2 Weeks	7th
AI Model Integration	4 Weeks	8th
Recommendation Engine	3 Weeks	8th
Search & Filtering	2 Weeks	8th
Testing & Debugging	3 Weeks	8th
Final Deployment	2 Weeks	8th

6.3 Development Tools

The development of Lawgorithm involved the use of modern tools and technologies to ensure efficiency, scalability, and performance.

Backend Framework: Django 6.0.1

Real-time Stack: Channels, Daphne, ASGI routing

AI/ML: Sentence-Transformers, scikit-learn cosine similarity, optional Google Gemini API

Databases: MySQL (local development), PostgreSQL + pgvector (Docker deployment path)

Auth/Social: django-allauth

DevOps/Runtime: Docker, Docker Compose

Language/Platform: Python 3.x, HTML/CSS/JS templates

6.4 Conclusion and Future Work / Extensions

Conclusion

The Lawgorithm system successfully demonstrates the integration of artificial intelligence with traditional web-based applications to solve real-world problems in the legal domain. The system provides an efficient and intelligent platform for matching clients with suitable lawyers based on case requirements. By combining NLP-based semantic analysis with rule-based filtering, the system ensures accurate, reliable, and explainable recommendations. The modular architecture and well-structured design make the system scalable and maintainable.

Future Work / Extensions

Although the system fulfills its core objectives, several enhancements can be implemented in the future:

- Mobile Application Development (Android/iOS)

- Enhanced real-time communication features such as voice and video calling between clients and lawyers
- Integration with Legal Databases and Court Records
- Advanced Machine Learning Models (e.g., Legal BERT)
- Voice-based Case Input using Speech Recognition
- Multilingual Support (Urdu and regional languages)
- Online Payment & Booking System
- AI-based Case Outcome Prediction
- Cloud Deployment (AWS, Azure, or GCP)

These enhancements can significantly improve the system's functionality, scalability, and real-world applicability

6.5 Limitations and Risk Analysis

6.5.1 Limitations

Despite achieving the core objectives of the Lawgorithm system, several limitations exist due to project scope, dataset size, and deployment constraints.

1. Limited Dataset Size: The system is currently tested on a relatively small dataset (approximately 50 lawyers and 200 case records). This limits the generalizability and robustness of the AI-based recommendation results. A larger and more diverse dataset would improve matching accuracy.

2. Dependence on Input Quality: The effectiveness of the recommendation engine heavily depends on the quality and completeness of client-provided case descriptions. Incomplete or vague inputs may lead to less accurate lawyer recommendations.

3. No Real Legal Validation: The system provides recommendations based on data-driven analysis and does not validate legal correctness or guarantee the suitability of a lawyer for a specific case. It should be considered as a decision-support tool rather than a final authority.

4. Local Deployment Constraint: The system is designed and deployed in a local environment as per academic requirements. Therefore, real-world factors such as large-scale traffic handling, cloud deployment challenges, and production-level security configurations are not fully tested.

5. Simplified AI Model Implementation: The system uses pre-trained models (e.g., Sentence Transformers) without domain-specific fine-tuning on legal datasets. This may limit the depth of semantic understanding compared to specialized legal AI systems.

6.5.2 Risk Analysis

The development and deployment of the Lawgorithm system involve certain risks that must be acknowledged and managed.

1. Recommendation Bias: The AI model may introduce bias based on available data, such as favouring lawyers with more recorded experience or certain locations, potentially leading to unfair recommendations.

2. Incorrect or Suboptimal Matches: There is a possibility that the system may recommend lawyers who are not the best fit for a case due to limitations in data, model understanding, or rule-based constraints.

3. Privacy and Data Security Risks: The system handles sensitive user data, including legal case details and personal information. In a production environment, inadequate security measures could lead to data breaches or unauthorized access.

4. Misuse of the Platform: Users may provide false information (e.g., fake lawyer profiles or misleading case descriptions), which can affect the reliability of the system and its recommendations.

5. System Dependency on AI Components: The recommendation engine depends on AI models and external configurations (e.g., optional Gemini API). Failures or misconfigurations in these components may impact system performance, although fallback mechanisms are implemented.

7. Quality Assurance

This chapter presents the quality assurance processes adopted during the development of the Lawgorithm system. It explains the testing strategies, methodologies, and validation techniques used to ensure that the system meets its functional and non-functional requirements. The chapter also includes a Requirement Traceability Matrix (RTM), which maps test cases to system requirements, ensuring complete coverage and verification of all functionalities.

7.1 Introduction

The quality assurance phase of the Lawgorithm system focuses on ensuring the reliability, accuracy, and performance of the application. A comprehensive testing strategy was implemented, including functional and system-level testing, to validate all components of the system. Since user satisfaction and system correctness are critical, structured test cases were designed and executed. To ensure that every functional requirement is properly validated, a Requirement Traceability Matrix (RTM) has been developed, which maps each requirement to its corresponding test cases, ensuring complete coverage and traceability.

7.2 Traceability Matrix

The Requirement Traceability Matrix (RTM) provides a mapping between system requirements and their corresponding test cases. It ensures that all functional requirements of the Lawgorithm system are tested and validated effectively. Each requirement is linked to one or more test cases, which verify its correctness and performance.

7.3 Detailed Test Cases

The following test cases are designed to validate the functional requirements of the Lawgorithm system. Each test case includes input conditions, expected outcomes, and system behaviour.

Test Case TC_1: User Registration

- **Requirement ID:** Req_1
- **Description:** Verify that a new user can successfully register
- **Input:**
 - Name: Anas
 - Email: anas@email.com
 - Password: 123456
 - Role: Client
- **Expected Output:**
 - User account is created successfully
 - Confirmation message displayed
- **Actual Output:**
 - Same as expected
- **Status:** Pass

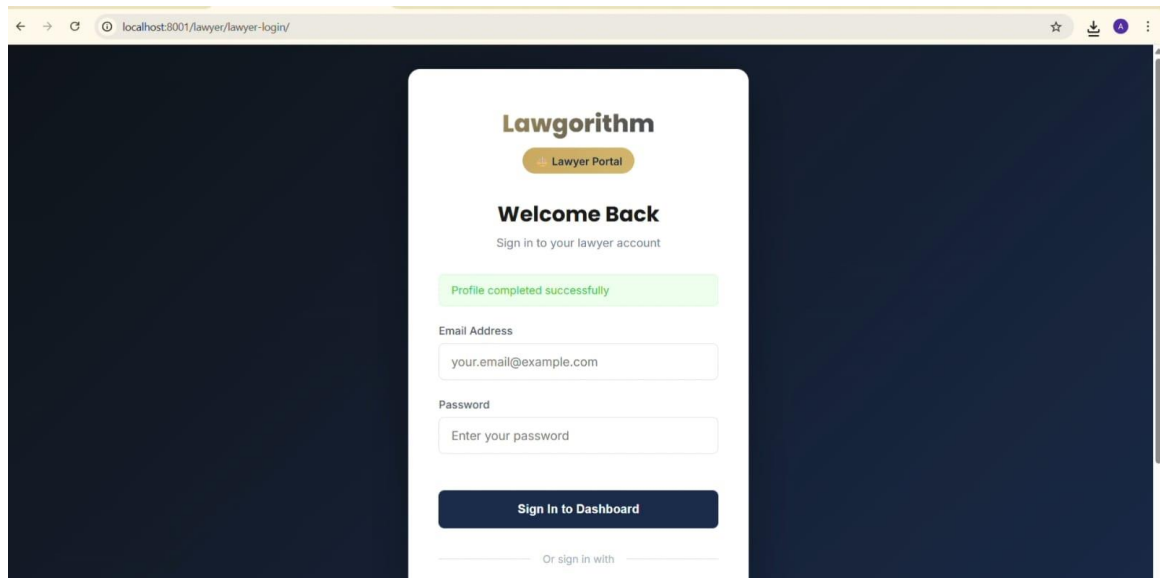


Figure 7.1: User registration

Test Case TC_2: User Login

- **Requirement ID:** Req_2
- **Description:** Verify that a registered user can log in
- **Input:**
 - Email: anas@email.com
 - Password: 123456
- **Expected Output:**
 - User is authenticated and redirected to dashboard
- **Actual Output:**
 - User is authenticated and redirected to dashboard
- **Status:** Pass

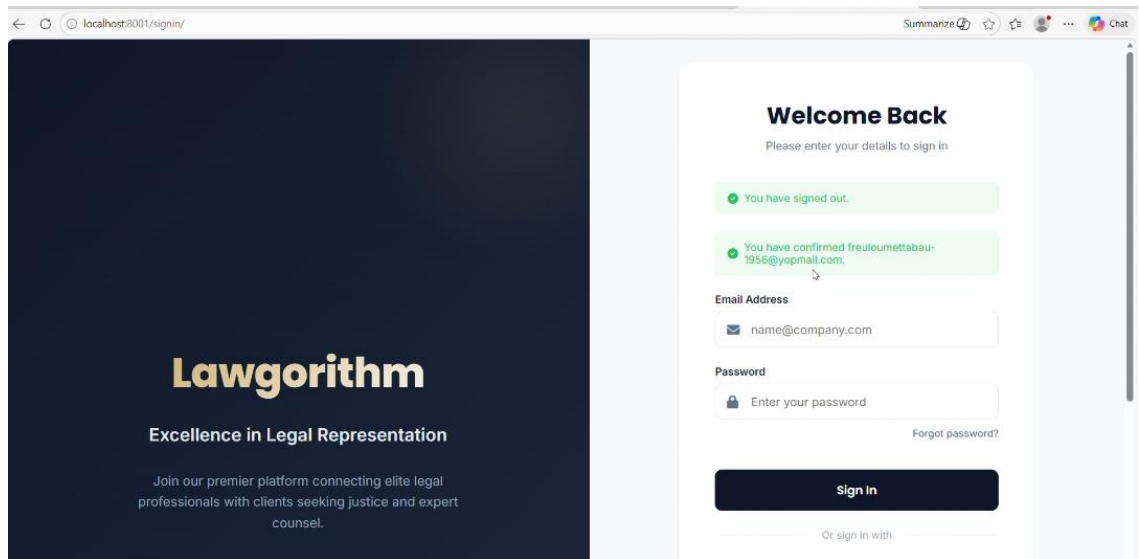


Figure 7.2: User login

Test Case TC_3: Case Submission

- **Requirement ID:** Req_3
- **Description:** Verify that a client can submit a case
- **Input:**
 - Case Title: Property Dispute
 - Description: Land ownership conflict
 - City: Lahore
 - Budget: Medium
- **Expected Output:**
 - Case is saved in database
 - Status set to “Submitted”
- **Actual Output:**
 - Case is saved in database
 - Status set to “Submitted”

- **Status:** Pass

The screenshot shows a web browser window with the URL 'localhost:8001/post-case/'. The page title is 'Post New Case'. A green notification bar at the top says 'Your case has been posted! Here are some recommended lawyers.' Below this is a 'Case Details' form with the following fields:

- Case Title: A text input field containing 'Untitled Case'.
- Category: A dropdown menu.
- Sub-category: A dropdown menu.
- Court Level: A dropdown menu.
- City: A dropdown menu.

Below the form is an 'Urgency' section with a text input field containing 'e.g., immediate action required'. At the bottom is a 'Case Description' section with a text input field. The left sidebar has the 'Lawgorithm' logo and navigation links: Overview, My Cases, Hired Lawyers, Find Lawyers, and Profile & Settings. The bottom of the sidebar shows the user's name 'naseem', 'Client Account', and a 'Logout' button.

Figure 7.3: Case submission

Test Case TC_4: Case Status Tracking

- **Requirement ID:** Req_4
- **Description:** Verify that case status updates correctly
- **Input:**
 - Case submitted
- **Expected Output:**
 - Status changes from “Submitted” → “Matched” after AI processing
- **Actual Output:**
 - Status changes from “Submitted” → “Matched” after AI processing
- **Status:** Pass

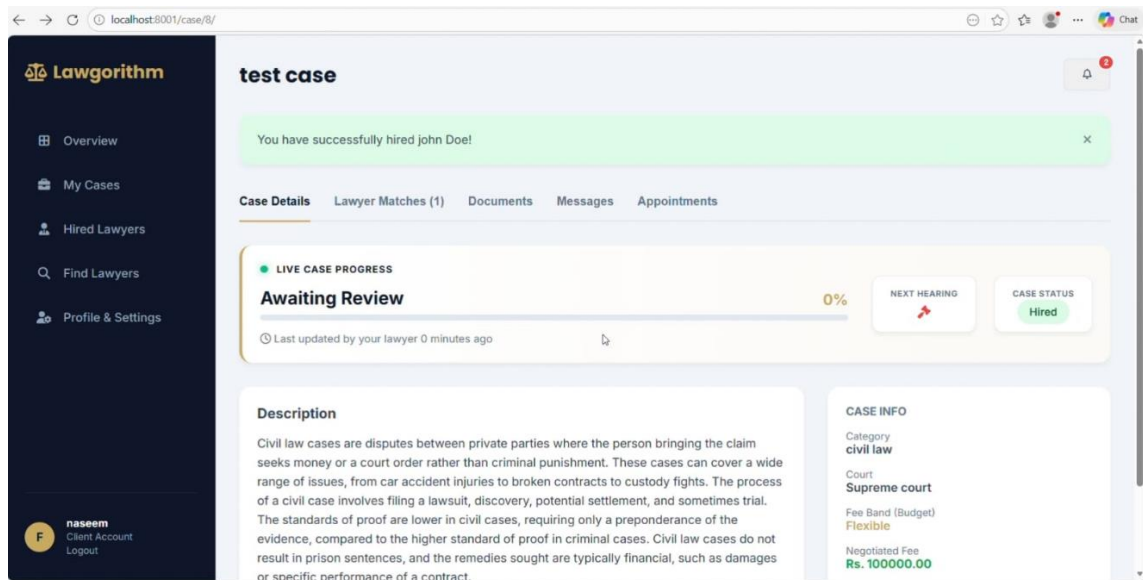


Figure 7.4: Case status tracking

Test Case TC_5: Lawyer Profile Creation

- **Requirement ID:** Req_5
- **Description:** Verify that a lawyer can create and update profile
- **Input:**
 - Name, specialization, experience, fee range
- **Expected Output:**
 - Profile is saved and visible in system
- **Actual Output:**
 - Profile is saved and visible in system
- **Status:** Pass

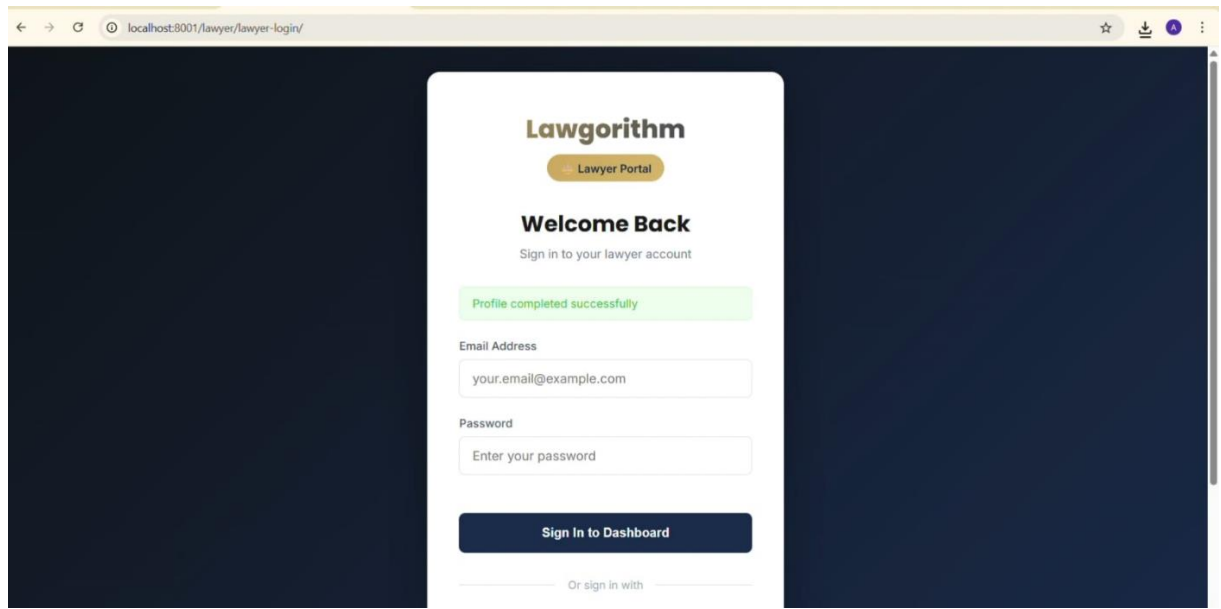


Figure 7.5: Lawyers profile creation

Test Case TC_6: AI Recommendation Generation

- **Requirement ID:** Req_6
- **Description:** Verify that system generates lawyer recommendations
- **Input:**
 - Case description submitted
- **Expected Output:**
 - List of recommended lawyers displayed
 - Each with Fit Score and explanation
- **Actual Output:**
 - List of recommended lawyers displayed
 - Each with Fit Score and explanation
- **Status:** Pass

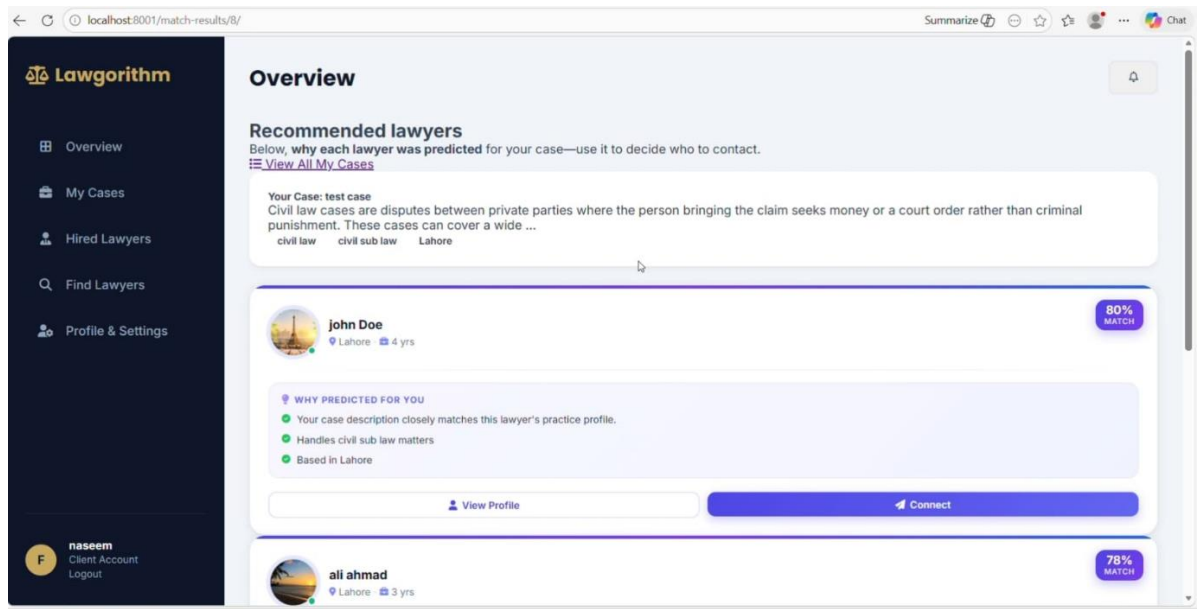


Figure 7.6: Recommended lawyers

Test Case TC_7: Fit Score Calculation

- **Requirement ID:** Req_7
- **Description:** Verify correctness of Fit Score calculation
- **Input:**
 - Case + Lawyer data
- **Expected Output:**
 - Fit Score generated between 0–100
 - Higher score for more relevant lawyer
- **Actual Output:**
 - Fit Score generated between 0–100
 - Higher score for more relevant lawyer
- **Status:** Pass

Test Case TC_8: Search and Filtering

- **Requirement ID:** Req_8
- **Description:** Verify search and filter functionality
- **Input:**
 - Search: “Criminal Lawyer”
 - Filter: City = Lahore
- **Expected Output:**
 - Relevant lawyers displayed based on criteria
- **Actual Output:**
 - Relevant lawyers displayed based on criteria
- **Status:** Pass

Test Case TC_9: Lawyer Verification (Admin)

- **Requirement ID:** Req_9
- **Description:** Verify admin can approve/reject lawyer
- **Input:**
 - Verification request submitted
- **Expected Output:**
 - Admin approves/rejects
 - Status updated accordingly
- **Actual Output:**
 - Admin approves/rejects
 - Status updated accordingly
- **Status:** Pass

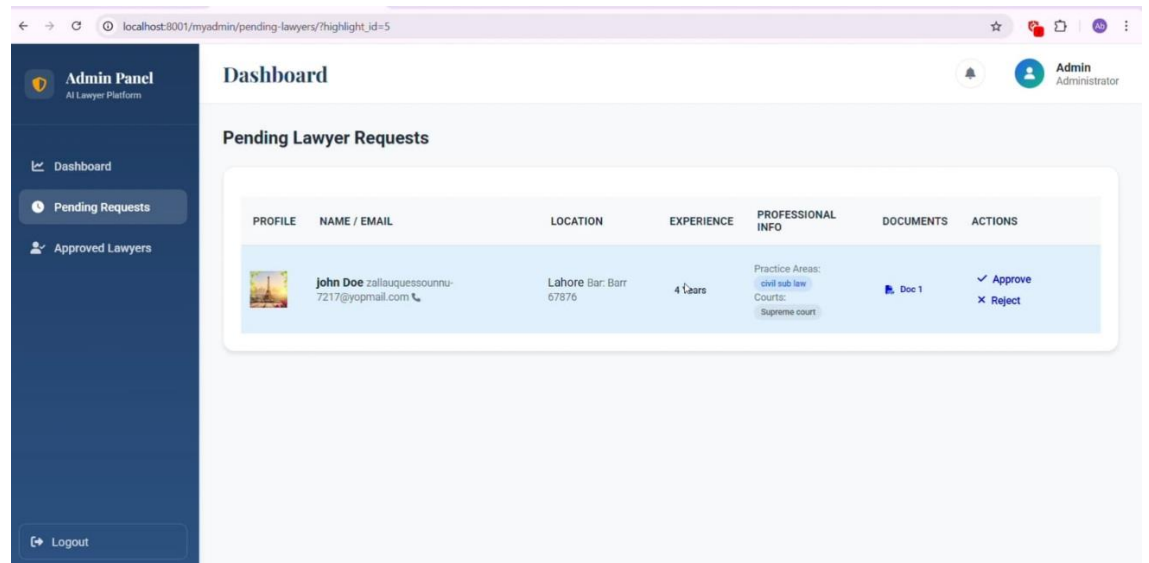


Figure 7.7: Lawyer verification

Test Case TC_10: Recommendation Display

- **Requirement ID:** Req_10
- **Description:** Verify recommendations are displayed correctly
- **Input:**
 - Completed matching process
- **Expected Output:**
 - Lawyers listed with Fit Score, reason tags, and profile link
- **Actual Output:**
 - Lawyers listed with Fit Score, reason tags, and profile link
- **Status:** Pass

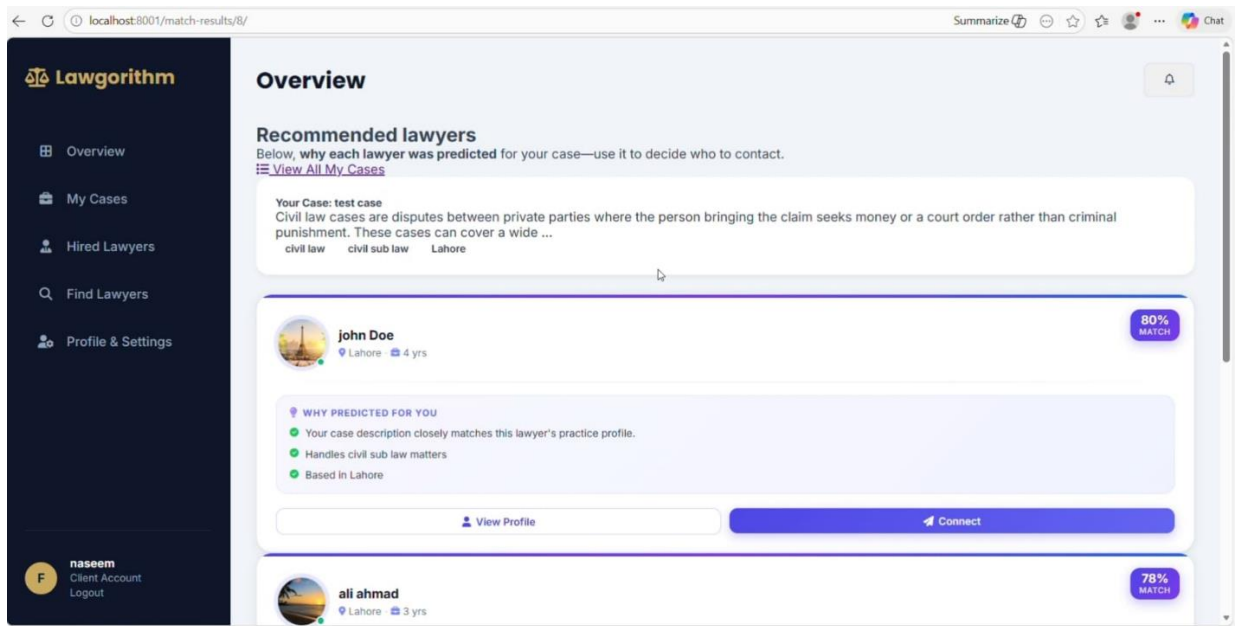


Figure 7.8: Recommendation display

7.4 Functional Requirements of Lawgorithm System

Req_1: User Registration

The system shall allow new users (clients and lawyers) to register by providing required details such as name, email, password, and role. The system must validate input data and create a secure user account.

Req_2: User Authentication (Login/Logout)

The system shall allow registered users to log in using valid credentials and access their respective dashboards. It shall also provide a logout mechanism to terminate user sessions securely.

Req_3: Case Submission

The system shall allow clients to submit legal cases by providing relevant details such as case title, description, category (practice area), location, and budget. The system shall store this information for further processing.

Req_4: Case Lifecycle Management

The system shall manage the lifecycle of a case, including status transitions such as Draft, Submitted, Matched, and Closed. Users shall be able to track the progress of their cases.

Req_5: Lawyer Profile Management

The system shall allow lawyers to create, update, and manage their professional profiles, including specialization, experience, qualifications, and fee structure.

Req_6: AI-Based Recommendation Engine

The system shall analyze submitted case details and generate a list of relevant lawyers using AI-based semantic matching and similarity techniques.

Req_7: Fit Score Calculation

The system shall compute a Fit Score (0–100) for each recommended lawyer based on semantic similarity and rule-based factors such as location, experience, and specialization.

Req_8: Search and Filtering

The system shall provide a search functionality that allows users to find lawyers based on keywords, specialization, and location, along with filtering options.

Req_9: Lawyer Verification (Admin Module)

The system shall allow administrators to verify and approve or reject lawyer profiles based on submitted credentials and documents.

Req_10: Recommendation Display

The system shall display recommended lawyers to clients along with their Fit Scores, profile details, and explanation of why they are recommended.

7.5 Requirement Traceability Matrix

Table 7.1: Requirement to Test Case Mapping

Req_ID	Requirement Description	TC_1	TC_2	TC_3	TC_4	TC_5	TC_6	TC_7	TC_8	TC_9	TC_10	Total TC
Req_1	User Registration	✓		✓			✓					3
Req_2	User Authentication		✓			✓						2
Req_3	Case Submission			✓								1
Req_4	Case Lifecycle Management				✓		✓					2
Req_5	Lawyer Profile					✓		✓				2

	Managem nt											
Req _6	AI Recomme ndation Engine						✓					1
Req _7	Fit Score Calculatio n					✓		✓				2
Req _8	Search & Filtering								✓			1
Req _9	Lawyer Verificatio n (Admin)									✓		1
Req _10	Recomme ndation Display										✓	1

7.6 Performance Evaluation

The performance of the Lawgorithm system was evaluated to measure its efficiency and responsiveness during execution. The evaluation focused on response time for key operations, including authentication, case submission, and recommendation generation.

AI Matching Evaluation

The effectiveness of the recommendation system was evaluated using standard information retrieval metrics.

Precision@K:

Measures the proportion of relevant lawyers in the top K recommendations. For example, in a test case where 4 out of top 5 recommended lawyers were relevant: $\text{Precision@5} = 0.8$ (80%)

Recall@K:

Measures the proportion of total relevant lawyers that were retrieved in the top K results. For example, if 6 lawyers were relevant and 4 were retrieved: $\text{Recall@5} = 0.67$ (67%)

Top-K Accuracy:

In experimental evaluation, the system achieved approximately **80% accuracy in Top-3 results**, meaning at least one relevant lawyer appeared in top 3 recommendations in most cases.

Test Environment

- Processor: Intel Core i5
- RAM: 8 GB
- Operating System: Windows 11
- Database: MySQL
- Dataset Size: ~50 lawyers, ~200 case records

Additional Evaluation Metrics:

The system also evaluates performance using:

- Mean Reciprocal Rank (MRR) to measure ranking quality
- Embedding coverage (%) to ensure all lawyers are represented
- P95 latency to evaluate worst-case response time
- Ranking consistency between LLM-based and fallback methods

Table 7.2: Performance Results

Operation	Average Response Time
User Login	0.8 seconds
Case Submission	1.2 seconds
Recommendation Generation	2.5 seconds
Search & Filtering	1.0 seconds

8. User Manual

This chapter provides a comprehensive guide for installing, configuring, and operating the Lawgorithm system. It includes system requirements, step-by-step installation instructions, and detailed guidelines for using all major functionalities. The purpose of this chapter is to enable users to effectively interact with the system without requiring technical expertise.

8.1 Introduction

This chapter presents the user manual for the Lawgorithm system, explaining how to install and operate the application. It provides step-by-step instructions for system setup and usage, ensuring that both technical and non-technical users can easily understand and interact with the system. The chapter also describes how different users (clients, lawyers, and administrators) can access and utilize system features.

8.2 Hardware/Software Requirements for the System

The following are the minimum requirements for installing and running the Lawgorithm system:

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: At least 10 GB free disk space
- Internet Connection: Required for web-based access

Software Requirements

- Operating System: Windows 10 / Linux / macOS
- Programming Language: Python 3.10 or above

- Web Framework: Django 5
- Database: MySQL
- Browser: Google Chrome / Microsoft Edge
- Libraries:
 - Sentence Transformers
 - FAISS (Facebook AI Similarity Search)
 - NumPy, Pandas

Development Tools (Optional for Developers)

- Visual Studio Code
- GitHub
- Postman

8.3 Installation Guide for Application

Follow the steps below to install and run the Lawgorithm system:

Step 1: Clone the Project

```
git clone https://github.com/your-repo/lawgorithm.git  
cd lawgorithm
```

Step 2: Create Virtual Environment

```
python -m venv venv  
venv\Scripts\activate (for Windows)
```

Step 3: Install Dependencies

pip install -r requirements.txt

Step 4: Configure Database

- Install MySQL and create a database:

CREATE DATABASE lawgorithm_db;

- Update database settings in settings.py

Step 5: Apply Migrations

python manage.py makemigrations

python manage.py migrate

Step 6: Create Superuser

python manage.py createsuperuser

Step 7: Run the Server

python manage.py runserver

Step 8: Access the Application

- Open browser and go to:

<http://127.0.0.1:8000/>

8.4 Operating Manual

This section explains how different users can operate the system.

1. User Registration & Login

- Open application homepage
- Click on **Register**
- Fill required details (name, email, password, role)
- Submit form
- Login using credentials



Figure 8.1: User Registration and login

2. Client Operations

Submit a Case

- Login as Client
- Click on **“Submit Case”**
- Enter case details (title, description, location, budget)
- Submit

View Recommendations

- After submission, system generates recommendations
- View list of lawyers with:
 - Fit Score
 - Profile details
 - Explanation (reason tags)

Track Case Status

- Navigate to **My Cases**
- View case progress (Submitted → Matched → Closed)

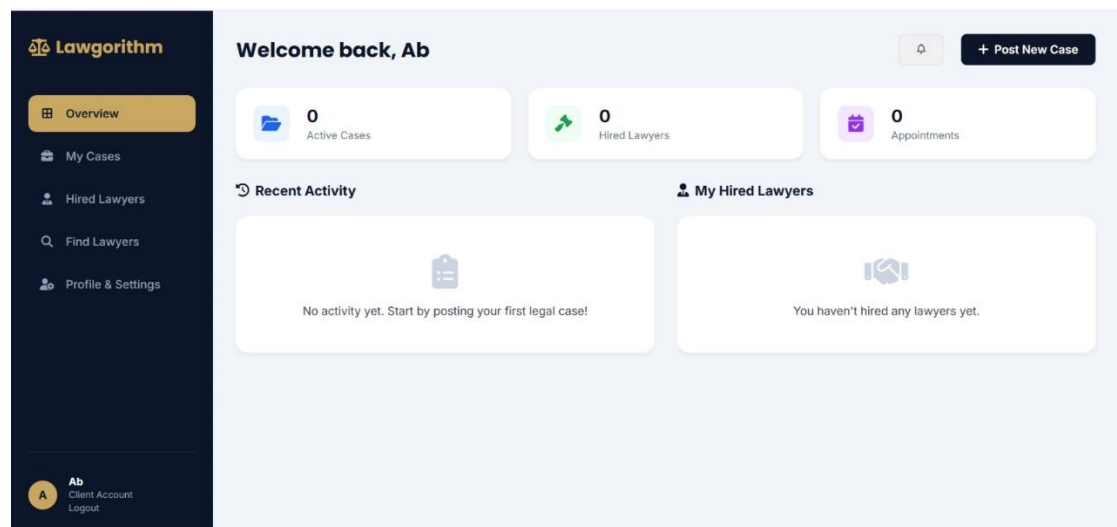


Figure 8.2: Client Operations

3. Lawyer Operations

Create/Update Profile

- Login as Lawyer
- Go to **Profile Section**

- Add specialization, experience, and fee details

Add Past Cases (Matters)

- Add previous case summaries
- Used for AI-based matching

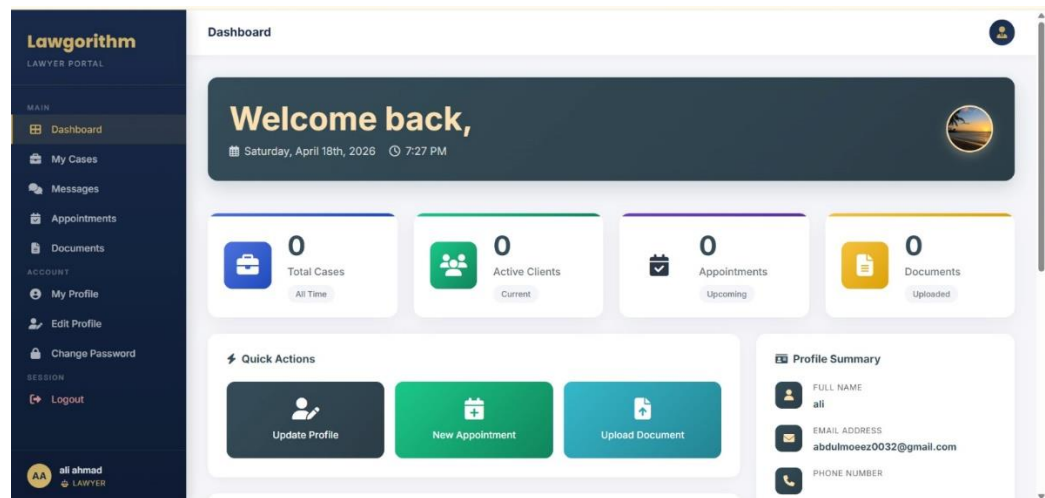


Figure 8.3: Lawyer Operations

4. Admin Operations

Verify Lawyers

- Login as Admin
- Open Admin Dashboard
- View pending verification requests
- Approve or Reject lawyers

Manage System Data

- Manage:
 - Practice areas
 - Locations
 - Users

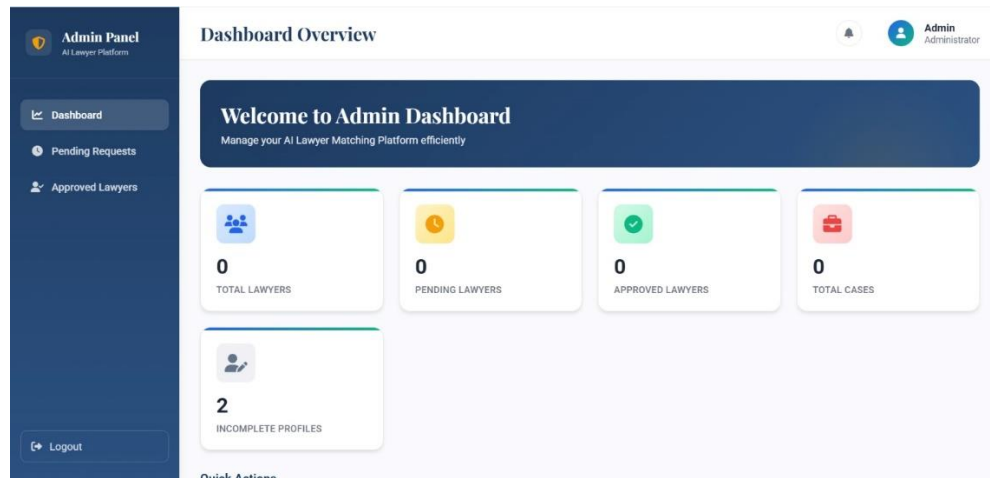


Figure 8.4: Admin Dashboard

5. Search & Filtering

- Use search bar to find lawyers
- Apply filters:
 - Location
 - Specialization

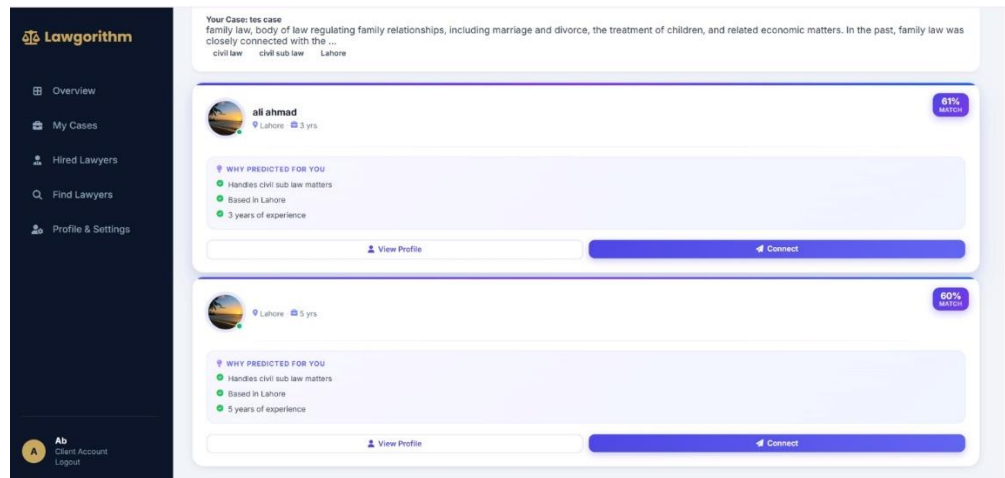


Figure 8.5: Search and filtering